Programmazione procedurale

Appunti

Marco Cerbella

Università degli Studi di Perugia – A.A. 2026/2027
Aggiornato al 3 novembre 2026

Indice

1 Algoritmi e diagrammi di flusso (Lezione 1 – 22 settembre 2026) 2

2 Esercizi sui diagrammi di flusso (Lezione 2 – 23 settembre 2026) 4

3 Il linguaggio C (Lezione 3 – 28 settembre 2026) 7

4 Struttura di un programma C e compilazione (Lezione 4 – 29 settembre 2026) 9

5 Errori di compilazione e di esecuzione (Lezione 5 – 30 settembre 2026) 12

6 Commenti, identificatori e parole chiave (Lezione 6 – 5 ottobre 2026) 13

7 Scope e binding (Lezione 7 – 6 ottobre 2026) 15

8 Tipi di dato (Lezione 8 – 7 ottobre 2026) 18

9 Costanti letterali (Lezione 9 – 12 ottobre 2026) 25

10 Conversioni di tipo (Lezione 10 – 13 ottobre 2026) 27

11 Espressioni, lvalue e operatori (Lezione 11 – 19 ottobre 2026) 32

12 Operatori bit a bit, altri operatori e sequence point (Lezione 12 – 19 ottobre 2026) 38

13 Istruzioni, blocchi e cicli (Lezione 13 – 26 ottobre 2026) 43

14 Selezione, salti e typedef (Lezione 14 – 26 ottobre 2026) 48

15 Array, stringhe e matrici (Lezione 15 – 2 novembre 2026) 54

16 Funzioni e funzione main (Lezione 16 – 3 novembre 2026) 59

1 Algoritmi e diagrammi di flusso

Lezione 1 – 22 settembre 2026

1.1 Algoritmi e diagrammi di flusso

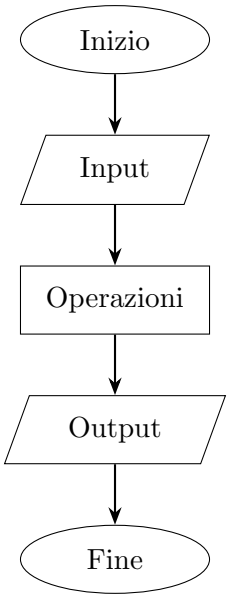
Definizione. Un *algoritmo* è una procedura che risolve un problema attraverso una sequenza finita di passi elementari e non ambigui. Spesso comporta la ripetizione di alcune operazioni.

Esempio. Il primo algoritmo celebre è quello di Euclide (*Elementi*, 300 a.C.) per il massimo comun divisore.

Definizione. Un *diagramma di flusso* (*flowchart*) è la rappresentazione grafica di un algoritmo. Le operazioni si leggono dall'alto verso il basso, seguendo le frecce. I simboli sono standardizzati (ANSI/ISO).

I simboli principali sono:

- **ellisse**: inizio e fine;
- **parallelogramma**: input e output (I/O);
- **rettangolo**: operazione (*processo*);
- **rombo**: decisione (condizione vero/falso).



Lo stesso schema in linguaggio *C*:

```c
// Importare le librerie necessarie
#include <stdio.h>

// Funzione principale
int main() {
    // Lettura dell'input e operazioni

    // Stampa del risultato
```

```
    return 0;
}
```

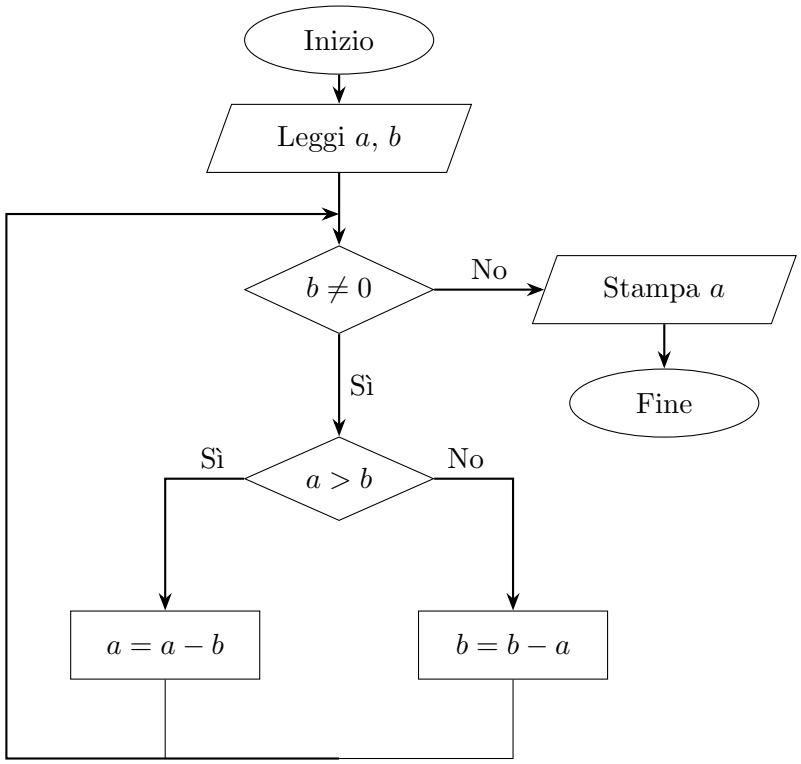
1.2 Programmare

Definizione. *Programmare* significa tradurre un algoritmo in una notazione, un linguaggio di programmazione, che un elaboratore può eseguire. Un *programma* è un insieme di istruzioni.

I passi sono: partire dal problema, *pensare all'algoritmo*, scegliere il linguaggio giusto e solo dopo programmare.

1.3 Esempio: massimo comun divisore

Algoritmo di Euclide (versione con sottrazioni): finché $b \neq 0$, si sostituisce il maggiore tra a e b con la differenza tra i due. Alla fine a contiene il MCD.



E il programma C corrispondente:

```c
#include <stdio.h>

int main() {
    int a, b;

    printf("Enter first positive integer: \n");
    scanf("%d", &a);
    printf("Enter second positive integer: \n");
    scanf("%d", &b);

```

```
    while (b != 0) {
        if (a > b)
            a = a - b;
        else
            b = b - a;
    }
    printf("GCD = %d\n", a);

    return 0;
}
```

Osservazione. Il ciclo `while` corrisponde al rombo $b \neq 0$ con la freccia che torna indietro; l'`if`/`else` corrisponde al secondo rombo.

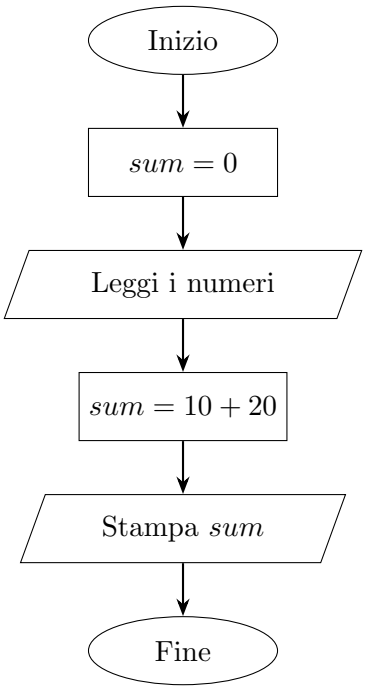
2 Esercizi sui diagrammi di flusso

Lezione 2 – 23 settembre 2026

2.1 Somma di 10 e 20

Si usa una variabile `sum` inizializzata a zero; si leggono i due numeri, si sommano salvando il risultato in `sum` e infine si stampa.

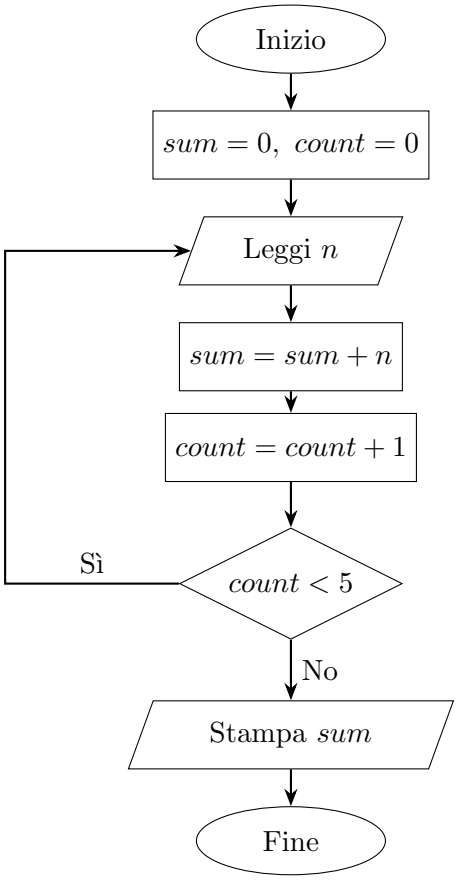
1. Inizializza $sum = 0$ (processo)
2. Leggi i numeri (I/O)
3. $sum = 10 + 20$ (processo)
4. Stampa sum (I/O)



2.2 Somma di 5 numeri

Servono due variabili: `sum` (il risultato) e `count` (quanti numeri sono stati letti). Si usa un *ciclo*: i passi si ripetono finché la condizione è vera.

1. $sum = 0$, $count = 0$ (processo)
2. Leggi n (I/O)
3. $sum = sum + n$ e $count = count + 1$ (processo)
4. $count < 5$? (decisione)
5. Se sì, torna al passo 2; altrimenti stampa sum (I/O)



Il programma C corrispondente:

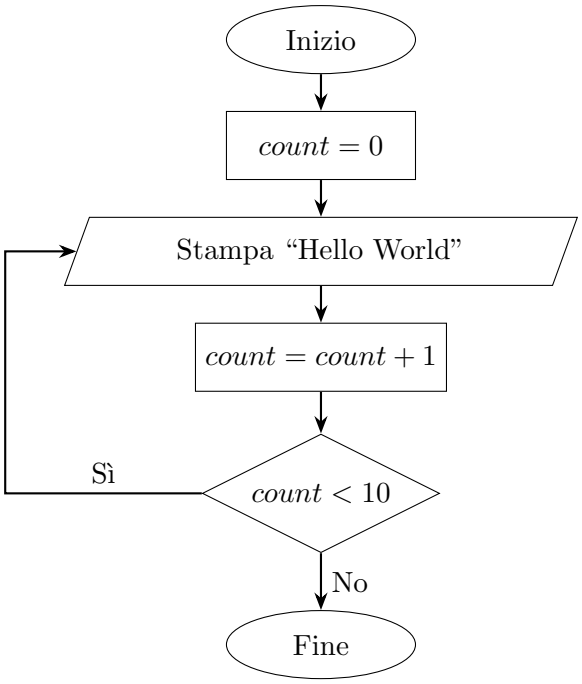
```c
#include <stdio.h>

int main() {
    int sum = 0, count = 0, n;
    while (count < 5) {
        scanf("%d", &n);
        sum = sum + n;
        count = count + 1;
    }
    printf("sum = %d\n", sum);
    return 0;
}
```

2.3 Stampare “Hello World” 10 volte

Anche qui si usa un ciclo con un contatore.

1. $count = 0$ (processo)
2. Stampa “Hello World” (I/O)
3. $count = count + 1$ (processo)
4. $count < 10$? Se sì torna al passo 2, altrimenti termina.




```c
#include <stdio.h>

int main() {
    int count = 0;
    do {
        printf("Hello World\n");
        count = count + 1;
    } while (count < 10);
    return 0;
}
```

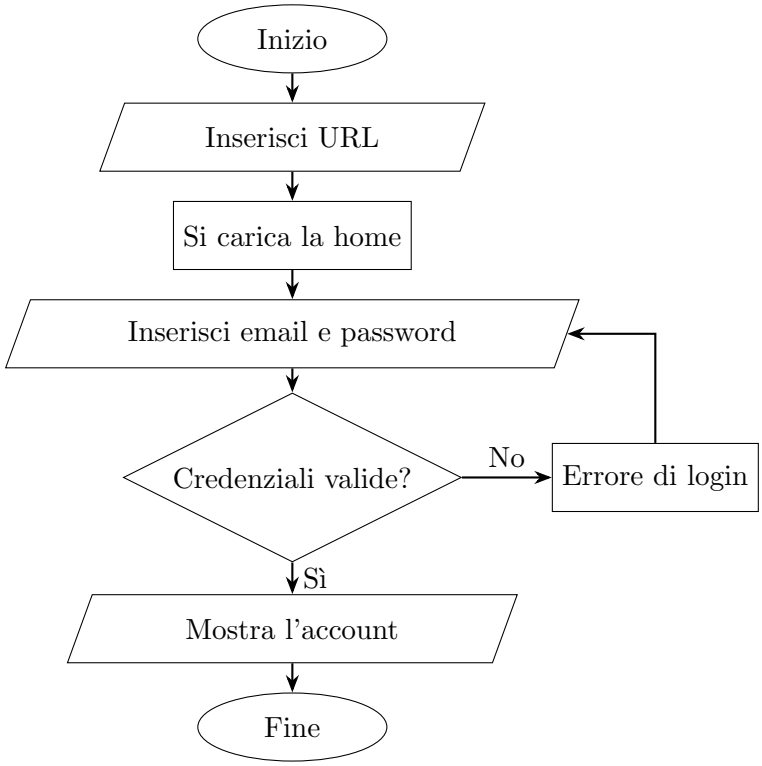
Osservazione. Il diagramma esegue la stampa *prima* di controllare la condizione, quindi il costrutto C fedele è `do ... while`. Con un `while` la condizione verrebbe controllata prima del corpo.

2.4 Login a un account (da risolvere liberamente)

Esercizio aperto: non esiste una sola soluzione. Una possibile sequenza:

1. Inserisci l’indirizzo del sito nel browser (I/O)
2. Si carica la home page (processo)
3. Inserisci email e password (I/O)
4. Credenziali valide? (decisione)

5. Se no: errore di login (processo) e torna al passo 3; se sì: mostra l'account (I/O) e termina.



3 Il linguaggio C

Lezione 3 – 28 settembre 2026

3.1 Linguaggi di programmazione

Definizione. Un *linguaggio di programmazione* è un linguaggio formale che specifica un insieme di istruzioni per un elaboratore. Si usa per realizzare programmi che implementano algoritmi. I linguaggi *ad alto livello* sono vicini al linguaggio umano e lontani dal linguaggio macchina.

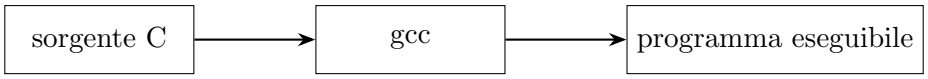
I *paradigmi di programmazione* classificano i linguaggi in base alle loro caratteristiche. Il C è *imperativo*: il programmatore indica alla macchina *come* modificare il proprio stato, ad esempio `a = a + 1;`.

3.2 Dal linguaggio macchina al C

- Il *linguaggio macchina* (sequenze di bit) è l'unico eseguito direttamente dal processore.
- L'*assembly* usa abbreviazioni mnemoniche per le operazioni, ma è legato alla macchina.
- I linguaggi *ad alto livello* sono indipendenti dalla macchina.

Definizione. Un *compilatore* è un programma di sistema che traduce il programma sorgente scritto in un linguaggio ad alto livello in un programma *target* equivalente (assembly/linguaggio macchina). Il programma target viene eseguito in un secondo momento.

In questo corso il compilatore è **gcc**.



3.3 Le fasi della compilazione con gcc

Quando si lancia `gcc`, il passaggio da sorgente a eseguibile non è un'unica operazione ma una catena di quattro fasi, ciascuna delle quali può essere fermata con una opzione.

Fase	**Comando**	**Risultato**	**Cosa fa**
Preprocessing	`gcc -E main.c -o main.i`	`main.i`	gestisce le righe che iniziano con `#`: sostituisce i `#define`, inserisce il contenuto degli `#include`, toglie i commenti. Il risultato è ancora codice C.
Compilazione	`gcc -S main.c`	`main.s`	traduce il C in *assembly*. Qui emergono gli errori di sintassi e di tipo.
Assemblaggio	`gcc -c main.c`	`main.o`	traduce l'assembly in codice macchina (*file oggetto*). Non è ancora eseguibile.
Linking	`gcc main.o -o prog`	`prog`	unisce i file oggetto e le librerie e produce l'eseguibile.

Osservazione. Un file oggetto contiene codice macchina ma le chiamate a funzioni definite in altri file o in libreria (ad esempio `printf`) non hanno ancora un indirizzo. Il *linker* li risolve. Un errore di linking tipico è `undefined reference to 'fun'`: la funzione è stata dichiarata ma non è stata definita in nessuno dei file collegati.

I file oggetto servono per compilare ogni file sorgente separatamente e unirli alla fine. Con due file `main.c` e `altro.c`:

```
gcc -c main.c
gcc -c altro.c
gcc -o prog main.o altro.o
```

oppure in un solo comando `gcc -o prog main.c altro.c`.

3.4 Caratteristiche del C

Il C è un linguaggio ad alto livello, *procedurale* e di uso generale, progettato (Dennis Ritchie, anni '70) per scrivere sistemi operativi con la massima indipendenza dall'hardware.

Pregi:

- veloce (compilato, vicino alla macchina) e portabile;
- linguaggio piccolo e maturo, con molti strumenti;
- accesso diretto alla memoria e alle funzionalità di basso livello.

Difficoltà:

- si può facilmente fare danni con accessi diretti alla memoria e puntatori;
- la memoria va gestita a mano;
- il codice è più verboso rispetto a linguaggi di scripting come Python.

Osservazione. Il linguaggio C in sé non ha istruzioni di input/output né di gestione della memoria dinamica: queste funzioni sono fornite dalla *libreria standard* (ad esempio `stdio.h`).

3.5 Standard

Il C è stato standardizzato più volte: C89/C90 (ANSI/ISO), C99, C11, C18. Con gcc si può scegliere lo standard:

```
gcc file.c -std=c11
```

3.6 Compilati e interpretati

Un linguaggio *compilato* (come C) viene analizzato a fondo dal traduttore prima dell'esecuzione, e quasi tutti i controlli semantici sono fatti a tempo di compilazione. In un linguaggio *interpretato* (come Python) l'interprete resta presente durante l'esecuzione e la maggior parte dei controlli è fatta a tempo di esecuzione.

Osservazione. Un interprete non produce necessariamente codice macchina: spesso esegue direttamente le istruzioni (Python, ad esempio, compila prima in *bytecode* e poi lo interpreta). Essere compilato o interpretato è inoltre una proprietà dell'*implementazione* più che del linguaggio. Un vantaggio del compilato è che molti errori (sintassi, tipi) sono segnalati prima che il programma giri; con l'interprete molti emergono solo quando la riga viene eseguita.

4 Struttura di un programma C e compilazione

Lezione 4 – 29 settembre 2026

4.1 Struttura di un programma

Gli elementi costitutivi di un programma C sono le *funzioni*, che possono chiamarsi a vicenda.

- Ogni funzione ha uno scopo preciso; le funzioni *non* si possono annidare.
- Una funzione contiene istruzioni eseguite in sequenza; le istruzioni si raggruppano in *blocchi*.
- Si possono usare funzioni della libreria standard (`printf`) o scriverne di proprie.
- Ogni programma deve definire almeno la funzione `main()`: è la prima funzione eseguita ed è il livello principale di controllo.

4.2 Hello World e compilazione con gcc

```c
#include <stdio.h>

int main() {
    printf("Hello World\n");
    return 0;
}
```

Salvato il codice in `test.c`, da terminale si compila (creando l'eseguibile `test`) e poi si esegue:

```
gcc -o test test.c
./test
```

Osservazione. Per il corso e per l'esame si compila *solo con gcc* da terminale (anche se si usa un editor che ha un terminale integrato). Senza `-o` il file eseguibile si chiama `a.out`.

4.3 Input e output: printf e scanf

Esempio (Somma di due interi)**.**

```c
#include <stdio.h>
int main() {
    int firstNumber, secondNumber, sumOfTwoNumbers;

    printf("Enter two integers: ");

    // gli interi inseriti vengono memorizzati con scanf()
    scanf("%d %d", &firstNumber, &secondNumber);

    // la somma viene salvata in sumOfTwoNumbers
    sumOfTwoNumbers = firstNumber + secondNumber;

    // stampa della somma
    printf("%d + %d = %d", firstNumber, secondNumber, sumOfTwoNumbers);

    return 0;
}
```

```
Enter two integers: 12
11
12 + 11 = 23
```

Esempio (Quoziente e resto)**.**

```c
#include <stdio.h>
int main() {
    int dividend, divisor, quotient, remainder;

    printf("Enter dividend: ");
    scanf("%d", &dividend);

    printf("Enter divisor: ");
    scanf("%d", &divisor);

    quotient = dividend / divisor;    // quoziente
    remainder = dividend % divisor;   // resto

```

```c
    printf("Quotient = %d\n", quotient);
    printf("Remainder = %d", remainder);

    return 0;
}
```

```
Enter dividend: 25
Enter divisor: 4
Quotient = 6
Remainder = 1
```

Osservazione. `%d` è il segnaposto per un intero; in `scanf` le variabili vanno precedute da `&`.

4.4 Programmazione procedurale e modulare

Definizione. La *programmazione procedurale* è un paradigma, derivato dalla programmazione strutturata, basato sul concetto di chiamata di procedura. Le procedure (o routine, sottoprogrammi, funzioni) contengono una serie di passi di calcolo da eseguire. Il C è imperativo e procedurale: le istruzioni sono raggruppate in funzioni.

Esempio. `mainfile.c`

```c
#include <stdio.h>
void myPrintHello();

int main() {
    myPrintHello();
    return(0);
}

void myPrintHello(void) {
    printf("Hello!\n");
    return;
}
```

Definizione. La *programmazione modulare* è una tecnica di progettazione che separa le funzionalità di un programma in *moduli* indipendenti e intercambiabili, ciascuno contenente tutto il necessario per realizzare un solo aspetto della funzionalità.

Ad esempio il codice si può dividere in file diversi.

`mainfile.c`:

```c
int main() {
    myPrintHello();
    return(0);
```

```
}
```

`hello.c`:

```c
#include <stdio.h>

void myPrintHello(void) {
    printf("Hello!\n");
    return;
}
```

e si compilano insieme:

```
gcc -o executable mainfile.c hello.c
```

5 Errori di compilazione e di esecuzione

Lezione 5 – 30 settembre 2026

5.1 Compile-time e run-time

La vita di un programmatore ha due momenti: il *compile-time*, quando si esegue `gcc`, e il *run-time*, quando si esegue il programma.

Definizione. Un *errore di compilazione* (*compile-time*) è un errore rilevato dal compilatore. Cause comuni: errori di sintassi, come un punto e virgola mancante o l'uso di una parola riservata (ad esempio `while`) come nome.

Definizione. Un *errore di esecuzione* (*run-time*) è un errore del programma che si verifica mentre il programma è in esecuzione.

5.2 Esempio di errore di compilazione

Nel programma del MCD manca il `;` dopo la seconda `scanf`:

```c
printf("Enter first positive integer: \n");
scanf("%d", &b)        // manca il ';'
```

Il compilatore segnala:

```
$ gcc -o euclid euclid.c
euclid.c:9:20: error: expected ';' after expression
   scanf("%d", &b)
                  ^
                  ;
1 error generated.
```

Il messaggio indica file, riga e colonna dell'errore: bisogna imparare a leggere l'output del compilatore per eliminare tutti gli errori di compilazione.

5.3 Esempio di errore di esecuzione

Il programma compila senza problemi, ma dereferenzia un puntatore nullo:

```c
#include <stdio.h>
int main() {
    int *a = NULL;

    printf("%d", *a);    // accesso a memoria non valida

    int i = 0;
    while (i < 4) {
        printf("Hello World");
        i++;
    }
    return 0;
}
```

```
$ gcc -o runtime_error runtime_error.c
$ ./runtime_error
Segmentation fault: 11
```

Osservazione. Eliminare gli errori di compilazione non basta: bisogna testare il programma con molti input diversi per eliminare (il più possibile) gli errori di esecuzione.

"Testing shows the presence, not the absence of bugs" – E. W. Dijkstra

6 Commenti, identificatori e parole chiave

Lezione 6 – 5 ottobre 2026

6.1 Commenti

In C ci sono due tipi di commenti:

- *a blocco*: iniziano con `/*` e finiscono con `*/`, possono occupare più righe;
- *di riga*: iniziano con `//` e finiscono a fine riga.

```c
double pi = 3.1415926536;   // Pi è 3,14
```

All'interno di una stringa o di una costante carattere i simboli `/*` e `//` non iniziano un commento:

```c
printf("Comments in C begin with /* or //.\n");
```

Osservazione. I commenti a blocco *non si possono annidare*: il primo `*/` chiude il commento aperto. Si possono invece usare per "spegnere" del codice che contiene commenti di riga.

Esempio. Qui il commento aperto alla riga 3 termina al primo `*/` (riga 6): il secondo `*/` è un errore di sintassi.

```
1 #include <stdio.h>
2 int main() {
3     int x = 4; /*
4     int y = 10; /*
5
6     printf("x = %d\n", x); */
7     printf("y = %d\n", y); */
8 }
```

6.1.1 Scrivere buoni commenti

Il codice si scrive per essere eseguito da un computer ma letto da persone: da noi stessi mentre lo scriviamo e dopo settimane o mesi, dai colleghi che devono integrarlo e da chi lo manterrà anni dopo.

- Evitare commenti ridondanti, come `i = i + 1; // incrementa i`.
- Non lasciare codice "eliminato" sotto forma di commento (va bene solo temporaneamente durante il debug).
- Evitare commenti decorati (riquadri di asterischi): sono difficili da mantenere.
- Un commento che chiarisce spesso segnala codice troppo complesso: meglio semplificare il codice, che deve essere *autoesplicativo*.

6.2 Caratteri

Il C definisce due insiemi di caratteri: quello *sorgente* (usabile nel codice) e quello di *esecuzione* (interpretabile dal programma in esecuzione); nella maggior parte delle implementazioni coincidono. I caratteri *spazio bianco* sono: spazio, `\f` (form feed), `\n` (a capo), `\t` (tab orizzontale), `\v` (tab verticale).

6.3 Identificatori

Definizione. Un *identificatore* è il nome di una variabile, funzione, macro, struttura o altro oggetto definito in un programma C. Può contenere:

- lettere `a--z` e `A--Z` (il C è *case-sensitive*);
- il carattere underscore _;
- cifre `0--9`, ma il primo carattere *non* può essere una cifra.

Esempio. Identificatori validi: `x`, `dollar`, `While`, `error_handler`, `scale64`.

Non validi: `1st_rank` (inizia con una cifra), `switch` (parola riservata), `y/n` e `x-ray` (caratteri non ammessi).

Non c'è un limite alla lunghezza, ma i compilatori considerano significativi solo i primi 31 o 63 caratteri. Molti nomi sono già usati dalla libreria standard (ad esempio `printf`) e non si possono riutilizzare per funzioni proprie o variabili globali.

6.4 Parole riservate

Le seguenti 44 parole chiave sono riservate in C (erano 32 in C89), hanno un significato preciso per il compilatore e non si possono usare come identificatori:

```
auto, break, case, char, const, continue, default, do, double, else, enum,
extern, float, for, goto, if, inline, int, long, register, restrict, return,
short, signed, sizeof, static, struct, switch, typedef, union, unsigned,
void, volatile, while, _Alignas, _Alignof, _Atomic, _Bool, _Complex, _Generic,
_Imaginary, _Noreturn, _Static_assert, _Thread_local
```

6.4.1 Scegliere i nomi

Un nome descrive l'oggetto, il suo comportamento e il suo uso: una variabile chiamata male può confondere molto.

- evitare nomi inutilmente lunghi;
- usare nomi diversi in ambiti diversi, per evitare confusione.

Esempio. Confrontare `a = b * c;` con `weekly_pay = hours_worked * pay_rate;`.

7 Scope e binding

Lezione 7 – 6 ottobre 2026

7.1 Scope degli identificatori

Definizione. Lo *scope* (ambito) di un identificatore è la parte del programma in cui l'identificatore è significativo, cioè in cui può essere "visto". Il tipo di scope dipende da dove l'identificatore viene dichiarato.

- **File scope**: l'identificatore è dichiarato fuori da tutti i blocchi e dalle liste di parametri (cioè fuori dalle funzioni). Si può usare dalla dichiarazione fino alla fine del file.
- **Block scope**: l'identificatore è dichiarato dentro un blocco. Si può usare dalla dichiarazione fino alla fine del blocco più piccolo che la contiene (spesso, ma non sempre, il corpo di una funzione).

Esempio.

```c
int p;                  // file scope

int fun2(void) {        // fun2: file scope
    int r = 3;          // r, s: block scope
    int s = r;
    while (r != s) {
        r--;
    }
    return r;
}
```

Osservazione. Non si possono avere due variabili con lo stesso nome nello stesso scope:

```c
int main() {
    int a = 5;
    float a = 6.5;   // errore: a già dichiarata
}
```

7.2 Shadowing

Si può riusare un identificatore in una dichiarazione *annidata* nello scope esistente; la nuova dichiarazione deve avere block scope, in un blocco contenuto in quello esterno. In questo caso la dichiarazione interna *nasconde* (*shadowing*) quella esterna, che non è visibile nello scope interno.

Esempio.

```c
int main() {
    int x = 1, y = 2, z = 3;
    printf(" x = %d, y = %d, z = %d \n", x, y, z);
    {
        int x = 10;
        float y = 20;
        printf(" x = %d, y = %f, z = %d \n", x, y, z);
        {
            int z = 100;
            printf(" x = %d, y = %f, z = %d \n", x, y, z);
        }
    }
    return 0;
}
```

```
x = 1, y = 2, z = 3
x = 10, y = 20.000000, z = 3
x = 10, y = 20.000000, z = 100
```

Esempio. Qui `x++` modifica la `x` esterna (non è ridichiarata), mentre `y++` modifica la `y` interna; uscendo dal blocco interno la `y` torna a valere 20.

```c
int main() {
    {
        int x = 10, y = 20;
        {
            printf("x = %d, y = %d\n", x, y);
            {
                int y = 40;
                x++;
                y++;
                printf("x = %d, y = %d\n", x, y);
            }
            printf("x = %d, y = %d\n", x, y);
```

```
        }
    }
    return 0;
}
```

```
x = 10, y = 20
x = 11, y = 41
x = 11, y = 20
```

Esempio. Fuori dal blocco in cui è dichiarata, una variabile non è accessibile:

```
int main() {
    {
        int x = 10;
    }
    {
        printf("%d", x);    // errore: x non visibile qui
    }
    return 0;
}
```

```
error: 'x' undeclared (first use in this function)
```

7.3 Binding statico

Definizione. Un *binding* è l'associazione tra un nome e l'oggetto che esso denota. Il *binding time* è il momento in cui il binding viene creato (più in generale, il momento in cui viene presa una decisione di implementazione). La regione testuale del programma in cui un binding è attivo è il suo scope.

In C lo scope di un binding è determinato *staticamente*, cioè a tempo di compilazione (*static scoping*), come si vede dal testo del programma e non dall'ordine delle chiamate a run-time. Altri linguaggi usano lo scoping *dinamico*: ad esempio in Perl `my` definisce una variabile locale a scope statico, `local` una a scope dinamico.

7.4 Esercizio

Esercizio. Qual è l'output del seguente programma?

```
#include <stdio.h>

int f1(void);
int f2(int x, int a);

int a;

```

```c
int main() {
    int a, b, c;

    a = 7;
    b = f1();
    c = f2(a, b);
    printf("%d %d %d\n", a, b, c);
}

int f1(void) {
    a = 12;
    printf("%d ", a);
    return (a + 5);
}

int f2(int x, int a) {
    printf("%d ", a);
    return (x * a);
}
```

Soluzione.

- In `main` la `a` locale (che vale 7) nasconde la `a` globale.
- `f1` non dichiara una propria `a`: per binding statico modifica quella *globale*, la pone a 12, stampa `12` e restituisce 17, quindi $b = 17$.
- In `f2(7, 17)` il parametro `a` vale 17: stampa `17` e restituisce $7 \cdot 17 = 119$, quindi $c = 119$.
- Infine `main` stampa `7 17 119`.

```
12 17 7 17 119
```

8 Tipi di dato

Lezione 8 – 7 ottobre 2026

8.1 Tipi e oggetti

I programmi devono memorizzare ed elaborare dati di tipo diverso, come interi e numeri in virgola mobile, in modi diversi: il compilatore deve quindi sapere che tipo di dato rappresenta un valore.

Definizione. In C un *oggetto* è una zona di memoria il cui contenuto può rappresentare valori. Gli oggetti che hanno un nome sono detti *variabili*. Il *tipo* di un oggetto determina:

- quanto spazio occupa in memoria;
- i valori che la variabile può assumere;
- le operazioni che si possono eseguire su di essa.

8.1.1 Tipi in C

- **Tipi di base**: interi (standard ed estesi) e floating-point (reali e complessi).
- **Tipi enumerati**.

- Il tipo `void`.
- **Tipi derivati**: puntatori, array, strutture, unioni, funzioni.

I tipi di base e i tipi enumerati formano i *tipi aritmetici*; i tipi aritmetici e i puntatori formano i *tipi scalari*; array e strutture sono detti *tipi aggregati*. Un *tipo funzione* descrive l'interfaccia di una funzione: il tipo del valore restituito ed eventualmente i tipi dei parametri.

8.2 Interi

I tipi interi con segno sono cinque (molti hanno sinonimi):

tipo	sinonimi
`signed char`	
`int`	`signed`, `signed int`
`short`	`short int`, `signed short`, `signed short int`
`long`	`long int`, `signed long`, `signed long int`
`long long` (C99)	`long long int`, `signed long long`, `signed long long int`

C definisce solo le dimensioni minime: `short` occupa almeno 2 byte, `long` almeno 4 e `long long` almeno 8. I tipi possono essere più grandi, ma devono rispettare l'ordine

$$\texttt{sizeof(short)} \leq \texttt{sizeof(int)} \leq \texttt{sizeof(long)} \leq \texttt{sizeof(long long)}.$$

8.2.1 Booleani

In C non esistono *true* e *false*: 0 è falso e qualsiasi valore diverso da 0 (ad esempio 1, −25, 123456) è vero.

```
1 if (3)                        if (0)
2     printf("YES\n");              printf("YES\n");
3 else                          else
4     printf("NO\n");               printf("NO\n");
```

Il primo `if` stampa `YES`, il secondo `NO`.

C99 ha introdotto il tipo intero senza segno `_Bool`: *vero* è codificato come 1, *falso* come 0. Includendo l'header `stdbool.h` si possono usare anche `bool`, `true` e `false`: `bool` è sinonimo di `_Bool` e `true`, `false` sono costanti uguali a 1 e 0.

8.2.2 Caratteri

Anche `char` è un tipo intero standard, ma il nome `char` è sinonimo di `signed char` oppure di `unsigned char` a seconda del compilatore. Occupa 1 byte e i caratteri corrispondono ai codici della tabella ASCII.

Con le variabili `char` si può fare aritmetica: sta al programmatore decidere se il numero contenuto è un codice di carattere o altro.

```
1 char ch = 'A';        // variabile di tipo char
2 printf("The character %c has the character code %d.\n", ch, ch);
3 printf("%c", ch + 1);
```

```
The character A has the character code 65.
B
```

8.3 Sistemi di numerazione

Definizione. Il sistema *binario* (base 2) rappresenta i numeri con due soli simboli, 0 e 1; ogni cifra è un *bit*. Quasi tutti i calcolatori lo usano internamente perché è semplice da realizzare con circuiti a porte logiche.

Definizione. Il sistema *esadecimale* (base 16) usa sedici simboli: le cifre 0–9 per i valori da zero a nove e le lettere `A, B, C, D, E, F` (o minuscole) per i valori da dieci a quindici. Ogni cifra esadecimale corrisponde a 4 bit (un *nibble*, metà di un byte da 8 bit): è una rappresentazione più leggibile dei valori binari. Il sistema *ottale* (base 8) usa le cifre 0–7.

Bin.	Esa.	Dec.	Bin.	Esa.	Dec.
0	0	0	1000	8	8
1	1	1	1001	9	9
10	2	2	1010	A	10
11	3	3	1011	B	11
100	4	4	1100	C	12
101	5	5	1101	D	13
110	6	6	1110	E	14
111	7	7	1111	F	15

8.3.1 Da binario/esadecimale a decimale

Un numero con cifre $d_n \dots d_1 d_0$ in base b vale $d_n b^n + d_{n-1} b^{n-1} + \cdots + d_1 b^1 + d_0 b^0$ (in base 2: $d_n 2^n + \cdots + d_1 2^1 + d_0 2^0 = N_{10}$).

Esempio. $1001_2 = 1 \cdot 2^3 + 0 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 = 9$ e $\texttt{002B}_{16} = 2 \cdot 16 + 11 = 43$.

8.3.2 Da decimale a binario/esadecimale

Si divide ripetutamente il numero per la base e si leggono i resti *dal basso verso l'alto*. Per l'ottale si usa lo stesso algoritmo con base 8.

Esempio. 156 in binario (divisioni per 2): i quozienti successivi sono $78, 39, 19, 9, 4, 2, 1, 0$ e i resti $0, 0, 1, 1, 1, 0, 0, 1$. Letti dal basso: $156 = 10011100_2$.

1565 in esadecimale (divisioni per 16): $1565 = 97 \cdot 16 + 13$ (resto $13 = \texttt{d}$), $97 = 6 \cdot 16 + 1$ (resto 1), $6 = 0 \cdot 16 + 6$ (resto 6). Quindi $1565 = \texttt{61d}_{16}$.

Con N bit si rappresentano i numeri da 0 a $2^N - 1$; con 8 bit (un byte) da 0 a 255 (da `0000 0000` a `1111 1111`). Servono però anche i numeri negativi, e per questo esistono rappresentazioni diverse: segno e modulo, complemento a due.

8.4 Rappresentazione degli interi in memoria

8.4.1 Segno e modulo

Un bit (di solito il più a sinistra) indica il segno: 0 per i positivi, 1 per i negativi; i bit restanti rappresentano il modulo. Ad esempio `1001 1000` vale -24 (mentre se si usasse solo per numeri positivi varrebbe 152).

Con n bit si rappresentano i valori da $-(2^{n-1}-1)$ a $2^{n-1}-1$, e ± 0: con 8 bit da -127 a $+127$.

> **Osservazione.** Problema: lo zero ha due rappresentazioni, `0000 0000` ($+0$) e `1000 0000` (-0). Una soluzione è il complemento a due.

8.4.2 Complemento a due

Valore binario	Complemento a due	Senza segno
`00000000`	0	0
`00000001`	1	1
⋮	⋮	⋮
`01111110`	126	126
`01111111`	127	127
`10000000`	-128	128
`10000001`	-127	129
`10000010`	-126	130
⋮	⋮	⋮
`11111110`	-2	254
`11111111`	-1	255

Nel complemento a due esiste un solo zero, `00000000`. Per cambiare segno a un numero (positivo o negativo) si **invertono tutti i bit e poi si somma 1**.

Esempio.

- `0000 0101` (valore 5); invertendo: `1111 1010`; sommando 1: `1111 1011`, che vale -5.
- Da `1111 1011`: invertendo `0000 0100`, sommando 1 `0000 0101`, cioè di nuovo 5.

Con n bit si rappresentano i valori da -2^{n-1} a $2^{n-1}-1$: non c'è "-0", quindi si rappresenta un numero negativo in più. Con 8 bit, da -128 (`1000 0000`) a $+127$ (`0111 1111`).

> **Osservazione.** La regola "inverti e somma 1" non funziona per -128, perché 128 non è rappresentabile con 8 bit in complemento a due. D'ora in poi si usa il complemento a due.

8.4.3 Endianness

> **Definizione.** L'*endianness* indica l'ordine usato per interpretare numericamente una sequenza di byte della memoria come una parola più grande (e l'ordine di trasmissione dei byte su un collegamento digitale). Le parole possono essere in formato *big-endian* o *little-endian*, a seconda che si parta dall'estremità più significativa (big end) o meno significativa (little end).

Esempi: i mainframe IBM z/Architecture e i Motorola 68000 usano big-endian, i processori Intel x86 little-endian.

La memoria è un array di byte; ogni byte ha il suo *indirizzo logico* (un numero positivo), cioè l'indirizzo al quale un elemento appare risiedere dal punto di vista di un programma in esecuzione. Per architetture a 64 bit il limite superiore è $2^{64} - 1$.

Esempio. Il numero 2 scritto su 4 byte, a partire dall'indirizzo 1500:

	1500	1501	1502	1503
big-endian	00	00	00	02
little-endian	02	00	00	00

Esempio. Sulla stringa di 4 bit `0010`, in complemento a due: in big-endian vale 2; in little-endian si leggono i bit in ordine inverso (`0100`) e vale 4. Sulla stringa `1010`: in big-endian vale -6, in little-endian (`0101`) vale 5.

8.5 Dimensione e limiti dei tipi

L'operatore `sizeof` restituisce la dimensione esatta in byte di un tipo o di una variabile: `sizeof (tipo)` oppure `sizeof espressione` (in questo caso conta il tipo dell'espressione).

```c
int iIndex;
iIndex = 1000;
// sizeof(int) e sizeof(iIndex) valgono entrambi 4
```

I limiti dei tipi interi per il proprio compilatore sono nell'header `limits.h`, che definisce macro come `INT_MIN`, `INT_MAX`, `UINT_MAX`, `CHAR_MIN`, `CHAR_MAX`.

```c
#include <stdio.h>
#include <limits.h>

int main() {
    printf(" char %zu %d %d\n", sizeof(char), CHAR_MIN, CHAR_MAX);
    printf(" int %zu %d %d\n", sizeof(int), INT_MIN, INT_MAX);
    return 0;
}
```

```
char 1 -128 127
int 4 -2147483648 2147483647
```

Osservazione. Il risultato di `sizeof` ha tipo `size_t`: per stamparlo è corretto `%zu` (nelle slide compare `%d`). I valori stampati dipendono dalla piattaforma (qui un caso tipico).

8.6 Numeri in virgola mobile

Per rappresentare i non interi, con la virgola in qualsiasi posizione, C ha tre tipi floating-point standard:

- `float`: precisione singola;
- `double`: precisione doppia;
- `long double`: precisione estesa.

8.6.1 Precisione

Un valore in virgola mobile si memorizza solo con una *precisione* limitata, determinata dal formato binario e dalla memoria usata. La precisione si esprime in cifre significative: ad esempio con sei cifre significative la conversione in un numero decimale di sei cifre restituisce le sei cifre originali. La posizione della virgola e gli zeri iniziali o finali non contano: 123 456 000 e 0,00123456 si possono entrambi memorizzare in un tipo con precisione a sei cifre.

In C le operazioni aritmetiche tra numeri in virgola mobile sono eseguite internamente con precisione double o superiore.

```
1 float height = 1.2345, width = 2.3456;   // precisione singola
2 double area = height * width;            // il calcolo è fatto in double
```

Se il risultato è assegnato a una variabile `float`, viene arrotondato se necessario. L'header `float.h` definisce macro come `FLT_MIN`, `FLT_MAX` e `FLT_DIG`, che indicano intervallo di valori e precisione del tipo `float`.

8.6.2 Notazione E e formato IEEE 754

La *notazione E* è la rappresentazione testuale della notazione scientifica: `1.234e+56` significa $1{,}234 \times 10^{56}$.

Nel formato *IEEE 754* ogni numero finito è descritto da tre interi: un segno s (zero o uno), un *significando* (o mantissa) c e un esponente q. Il valore è

$$(-1)^s \times c \times b^q,$$

dove b è la base (2 o 10). Ad esempio con base 10, segno 1 (negativo), significando 12345 ed esponente -3 il valore è $-12{,}345$.

Il formato 754 a *precisione singola* usa: 1 bit di segno, 8 bit di esponente, 23 bit di significando. Il valore è

$$(-1)^{\text{segno}} \times \left(1 + \sum_{i=1}^{23} b_{23-i}\, 2^{-i}\right) \times 2^{(e-127)}.$$

Esempio. Con segno 0, esponente `01111100` ($e = 124$) e frazione `010...0` (cioè $b_{21} = 1$, tutti gli altri bit nulli) si ha $1 + 2^{-2} = 1{,}25$ e $2^{124-127} = 2^{-3}$, quindi

$$\text{valore} = (+1) \times 1{,}25 \times 2^{-3} = +0{,}15625.$$

Con la *precisione doppia* (`double`) si usano 1 bit di segno, 11 di esponente e 52 di frazione; la *precisione estesa* (`long double`, 80 bit) ha 1 bit di segno, 15 di esponente, 1 bit di parte intera e 63 di frazione.

8.6.3 Errori di arrotondamento

L'errore di arrotondamento è intrinseco nel calcolo in virgola mobile.

```
1 #include <stdio.h>
2
3 int main() {
4     int a = 16777217;
```

```
    float b = a;
    printf("%f\n", b);
}
```

```
16777216.000000
```

Il valore $16777217 = 2^{24} + 1$ non è rappresentabile esattamente con un `float`. Allo stesso modo conviene tenere in `double` anche il risultato di un prodotto come `height * width`: salvarlo in un `float` fa perdere precisione.

Osservazione. Il modo più semplice per evitare di accumulare errori è usare numeri in virgola mobile ad alta precisione, cioè `double` al posto di `float`. Sulle CPU moderne non c'è (quasi) penalizzazione di tempo, ma un `double` occupa il doppio della memoria.

8.7 Tipi enumerati

Le *enumerazioni* sono tipi interi definiti dal programmatore. La definizione inizia con la parola chiave `enum`, eventualmente seguita da un identificatore, e contiene l'elenco dei possibili valori con un nome per ciascuno:

```
enum [identificatore] { lista-enumeratori };
```

Esempio.

```
enum color { black, red, green, yellow, blue, white = 7, gray };
```

Le costanti elencate valgono $0, 1, 2, 3, 4, 7, 8$ (se non si assegna un valore, si continua dal precedente più uno).

```
enum color fgColor = blue,          // due variabili
           bgColor = yellow;        // di tipo enum color

void setFgColor(enum color fgc);    // funzione con parametro enum color
```

Con le variabili di tipo enumerato si può fare aritmetica ordinaria (`red + red` vale 2). Costanti diverse della stessa enumerazione possono avere lo stesso valore:

```
enum signals { OFF, ON, STOP = 0, GO = 1, CLOSED = 0, OPEN = 1 };
```

Esempio.

```
#include <stdio.h>

```

```c
enum week { sunday, monday, tuesday, wednesday, thursday, friday, saturday };

int main() {
    enum week today;
    today = wednesday;
    printf("Day %d", today + 1);
    return 0;
}
```

```
Day 4
```

Osservazione. Conviene usare sempre un'enumerazione quando una variabile (in particolare un parametro di funzione) può assumere solo uno tra pochi valori possibili: si evitano errori dovuti a costanti non valide, si documentano i valori ammessi ed è più mnemonico degli interi.

8.8 Il tipo void

Il tipo `void` indica che non è disponibile alcun valore. Non si possono dichiarare variabili o costanti di questo tipo, ma si usa:

- nelle dichiarazioni di funzioni;
- nelle espressioni `void`;
- nei puntatori a `void` (si vedranno con i puntatori).

Una funzione che non restituisce nulla ha tipo `void`: `void error(int a) {}`. `void` nella lista dei parametri di un prototipo indica che la funzione non ha parametri: `void printMenu(void) {}`. Il compilatore segnala un errore se si chiama `printMenu(3)`.

9 Costanti letterali

Lezione 9 – 12 ottobre 2026

9.1 Costanti letterali

Definizione. Un *literal* (costante letterale) è un token che denota un valore fisso, che può essere un intero, un numero in virgola mobile, un carattere o una stringa. Il tipo di un literal è determinato dal suo valore e dalla sua notazione.

9.2 Costanti intere

Una costante intera si può scrivere come un normale numero decimale oppure in notazione ottale o esadecimale. La notazione si specifica con un *prefisso*.

- **Decimale**: inizia con una cifra diversa da zero.
- **Ottale** (base 8, cifre $0, \ldots, 7$): inizia con uno zero iniziale. Il valore è $c_n 8^n + c_{n-1} 8^{n-1} + \cdots + c_1 8^1 + c_0 8^0$.
- **Esadecimale**: inizia con il prefisso `0x` o `0X`. Le cifre da A a F possono essere maiuscole o minuscole.

Esempio.

- $047 = 4 \cdot 8^1 + 7 \cdot 8^0 = 39$.
- `0xff`, `0Xff`, `0xFF` e `0XFF` valgono tutte $15 \cdot 16^1 + 15 \cdot 16^0 = 255$.

9.2.1 Tipo delle costanti intere

Il tipo di una costante è determinato nel momento in cui ne viene definito il valore. Le costanti intere degli esempi precedenti hanno di solito tipo `int`. Se il valore è troppo grande, il compilatore assegna il primo tipo, in una gerarchia, abbastanza grande da rappresentarlo. Ad esempio, la costante decimale `50000` ha tipo `int`, perché il massimo valore possibile per uno `short` (2 byte) è $32\,767 = 2^{15} - 1$.

Si può influenzare esplicitamente il tipo di una costante con un *suffisso*:

- `512U`: `unsigned int`;
- `0Xf0fUL`: `unsigned long`;
- `07771l`: `long long`;
- `123uLL`: `unsigned long long`.

Maiuscole e minuscole sono equivalenti (`ll` e `LL` sono la stessa cosa).

```
1 int a = 512U;
2 int b = 1LL;
3 int c = 7UL;
```

9.3 Costanti in virgola mobile

Le costanti in virgola mobile si scrivono in notazione decimale o esadecimale. Una costante è una sequenza di cifre decimali contenente un punto decimale. Si può moltiplicare il valore per una potenza di 10, come nella notazione scientifica: la potenza di 10 si indica con un esponente introdotto dalla lettera `e` o `E`.

Esempio. `2.34E5` vale $2.34 \cdot 10^5$ e `67e-12` vale $67.0 \cdot 10^{-12}$.

Il punto decimale può anche essere il primo o l'ultimo carattere: `10.` e `.234E6` sono numeri validi. Il numero `10` senza punto decimale è invece una costante *intera*, non in virgola mobile.

9.3.1 Forzare il tipo

Il tipo di default di una costante in virgola mobile è `double`. Con il suffisso `F` o `f` si assegna il tipo `float`; con il suffisso `L` o `l` il tipo `long double`.

9.4 Costanti carattere

Una costante carattere è un carattere racchiuso tra apici singoli: `'a'`, `'0'`, `'*'`. Per rappresentare l'apice, il backslash e il ritorno a capo si usano `'\''`, `'\\'` e `'\n'`. Le costanti carattere hanno tipo `int`.

9.4.1 Sequenze di escape

Sequenza	Valore del carattere	Azione sul dispositivo di output
`\'`	apice singolo (')	stampa il carattere
`\"`	virgolette (")	stampa il carattere
`\?`	punto interrogativo	stampa il carattere
`\\`	backslash	stampa il carattere
`\a`	alert	produce un segnale acustico o visivo
`\b`	backspace	riporta la posizione attiva indietro di un carattere
`\f`	form feed	porta la posizione all'inizio della pagina successiva
`\n`	line feed	porta la posizione all'inizio della riga successiva
`\r`	carriage return	porta la posizione all'inizio della riga corrente

9.5 Stringhe letterali

Una *stringa letterale* è una sequenza di caratteri (e/o sequenze di escape) racchiusa tra virgolette doppie, ad esempio `"Hello world!\n"`. Come le costanti carattere, può contenere tutti i caratteri del set di caratteri sorgente.

Una stringa letterale è un *array statico di char* che contiene i codici dei caratteri seguiti dal terminatore di stringa, il carattere nullo `\0`. La stringa vuota `""` occupa esattamente un byte di memoria, che contiene il carattere nullo di terminazione.

```
1 char helloWorld[128] = "Hello World!\n";
2 printf("Print the string: %s\n", helloWorld);
```

Riferimenti

Sul libro: sezione 15.6 (pagina 339) e pagina 46, sezione 9.10.

10 Conversioni di tipo

Lezione 10 – 13 ottobre 2026

10.1 Conversioni

Operandi di tipo diverso si possono combinare in una stessa operazione.

```
1 double dVar = 2.5;
2 dVar = dVar * 3;
3 if (dVar < 10L) { /* ... */ }
```

Quando gli operandi hanno tipi diversi, il compilatore cerca di convertirli a un tipo uniforme prima di eseguire l'operazione.

10.1.1 Conversioni implicite

Il compilatore applica conversioni di tipo implicite quando

- gli operandi hanno tipi non corrispondenti;
- si chiama una funzione con un argomento il cui tipo non corrisponde a quello del parametro;

- si inizializzano variabili o si assegnano loro valori.

Se la conversione necessaria non è possibile il compilatore segnala un errore; altre volte segnala un warning.

10.1.2 Operatore di cast

Si può convertire un valore da un tipo a un altro anche *esplicitamente*, con l'operatore di cast:

```
(nome_tipo) espressione
```

```
1 int sum = 10, count = 3;
2 double mean = (double) sum / count;   // 3.3333
```

Il valore di `sum` viene prima convertito (esplicitamente) in `double`; il compilatore deve poi convertire implicitamente il divisore, il valore di `count`, allo stesso tipo prima di eseguire la divisione.

Osservazione. Conviene usare sempre il cast quando c'è la possibilità di perdere informazione. I cast espliciti evitano i warning del compilatore.

10.2 Conversione dei tipi aritmetici

10.2.1 Gerarchia dei tipi

Quando gli operandi aritmetici hanno tipi diversi, la conversione implicita è governata dal *rango di conversione* dei tipi:

- due tipi interi senza segno qualsiasi hanno ranghi diversi: se uno è più largo dell'altro ha rango più alto;
- ogni tipo intero con segno ha lo stesso rango del corrispondente tipo senza segno;
- i tipi interi standard sono ordinati: `_Bool` < `char` < `short` < `int` < `long` < `long long`;
- i tipi in virgola mobile sono ordinati: `float` < `double` < `long double`;
- il tipo in virgola mobile di rango più basso, `float`, ha rango più alto di qualunque tipo intero;
- gli `enum` hanno lo stesso rango di `int`.

10.2.2 Promozione intera

In qualunque espressione si può usare un valore di tipo con rango inferiore a `int` al posto di un operando di tipo `int` o `unsigned int`. In questi casi il compilatore applica la *promozione intera*: ogni operando di rango inferiore a `int` viene convertito automaticamente in `int`, purché `int` sia in grado di rappresentare tutti i valori del tipo originale; altrimenti viene convertito in `unsigned int`. Le operazioni nella CPU vengono eseguite su almeno 4 byte.

Esempio.

```
1 #include <stdio.h>
2 int main()
3 {
4     char a = 30, b = 40, c = 10;
```

```
    char d = (a * b) / c;
    printf("%d ", d);
    return 0;
}
```

```
120
```

A prima vista `(a*b)/c` sembra causare un overflow aritmetico: i `char` con segno vanno da -128 a 127 (nella maggior parte dei compilatori) e il valore di `(a*b)` è 1200. Ma sui tipi `char` avviene la promozione intera, quindi si ottiene il risultato corretto senza overflow.

10.2.3 Conversioni aritmetiche usuali

Le *conversioni aritmetiche usuali* sono le conversioni implicite applicate automaticamente agli operandi di tipo aritmetico diverso per la maggior parte degli operatori. Si applicano implicitamente a

- gli operatori aritmetici a due operandi: `*`, `/`, `%`, `+` e `-`;
- gli operatori relazionali e di uguaglianza: `<`, `<=`, `>`, `>=`, `==` e `!=`.

Esempio.

```
int x = 0;
int i = -1;
unsigned int limit = 200U;
long n = 30L;

if (i < limit)
    x = limit * n;

printf("%d\n", x);
```

```
0
```

Le conversioni aritmetiche usuali si applicano così:

- Se uno degli operandi ha tipo in virgola mobile, l'operando di rango più basso viene convertito a un tipo con lo stesso rango dell'altro. I tipi reali si convertono solo in tipi reali.
- Se entrambi gli operandi sono interi, si esegue prima la promozione intera su entrambi. Se dopo la promozione i tipi sono ancora diversi, si procede così:
  - **Regola 1**: se un operando ha un tipo *senza segno* T con rango almeno pari a quello del tipo dell'altro operando, l'altro operando viene convertito al tipo senza segno T.
  - **Regola 2**: altrimenti, un operando ha un tipo *con segno* T con rango più alto di quello dell'altro. L'altro operando viene convertito al tipo con segno T solo se T è in grado di rappresentare tutti i valori del suo tipo precedente; se non lo è, entrambi gli operandi vengono convertiti al tipo senza segno corrispondente a T.

Esempio (Applicazione delle regole)**.** Nell'esempio sopra, per valutare il confronto `i < limit` il valore di `i`, -1, viene convertito a `unsigned int` (Regola 1: `limit` è senza segno e ha rango almeno pari a `int`). Il risultato è un numero positivo molto grande, quindi la condizione dell'`if` è *falsa* e `x` resta 0.

In `limit * n` (Regola 2) `limit` viene convertito al tipo di `n`, `long`, se l'intervallo dei valori di `long` contiene tutto l'intervallo di `unsigned int`. Altrimenti (ad esempio se `int` e `long` hanno entrambi 32 bit) entrambi i fattori vengono convertiti a `unsigned long`.

Come si rappresenta -1 in un `int` (little endian, complemento a due)? Con 32 bit tutti a 1: `11111111111111111111111111111111`. Un `unsigned int` ha 32 bit (in quel compilatore), quindi -1 viene convertito implicitamente in `unsigned int` e il suo valore diventa 4 294 967 295.

10.3 Altre conversioni implicite

Il compilatore converte automaticamente i valori aritmetici anche nei casi seguenti:

- nelle assegnazioni e nelle inizializzazioni, il valore dell'operando di destra viene sempre convertito al tipo dell'operando di sinistra;
- nelle chiamate di funzione, gli argomenti vengono convertiti ai tipi dei parametri corrispondenti;
- nelle istruzioni `return`, il valore dell'espressione viene convertito al tipo restituito dalla funzione.

In un'*assegnazione composta*, come `x += 2.5` (cioè `x = x + 2.5`), i valori di entrambi gli operandi sono prima soggetti alle conversioni aritmetiche usuali; poi il risultato dell'operazione viene convertito, come in una assegnazione semplice, al tipo dell'operando di sinistra. Se `x` è `int`, viene convertito in `double` e poi il risultato `x + 2.5` (di tipo `double`) viene riconvertito in `int`.

Esempio.

```
#include <math.h>   // dichiara double sqrt(double)
int i = 7;
float x = 0.5;      // la costante viene convertita da double a float

i = x;              // il valore di x viene convertito da float a int

x += 2.5;           // prima dell'addizione x viene convertito in double;
                    // la somma viene poi convertita in float per l'assegnazione

x = sqrt(i);        // l'argomento viene convertito da int a double;
                    // il valore restituito da double a float

long my_func() {    // la costante 0 viene convertita a long,
    /* ... */       // il tipo restituito dalla funzione
    return 0;
}
```

10.4 Conversioni verso tipi interi

10.4.1 Verso tipi interi senza segno

I valori interi sono sempre preservati se rientrano nell'intervallo del nuovo tipo senza segno, cioè tra 0 e `Utype_MAX`. Per i valori fuori intervallo, il valore dopo la conversione si ottiene sommando

o sottraendo `Utype_MAX` + 1 tante volte quante servono perché il risultato rientri nell'intervallo del nuovo tipo.

```
unsigned short n = 1000;    // 1000 rientra nell'intervallo di unsigned short: ok
n = -1;                     // il valore -1 deve essere convertito
```

Si ha $-1 + (\texttt{USHRT_MAX} + 1) = \texttt{USHRT_MAX}$, quindi l'ultima istruzione equivale a `n = USHRT_MAX;`.

10.4.2 Da virgola mobile a interi

Per convertire un numero reale in virgola mobile in un tipo intero (con o senza segno) il compilatore scarta la parte frazionaria. Se la parte intera rimanente è fuori dall'intervallo del nuovo tipo, il risultato della conversione è *indefinito*.

```
double x = 2.9;
unsigned long n = x;    // la parte frazionaria di x viene persa: n = 2
```

10.4.3 Verso tipi interi con segno

Il problema del superamento dell'intervallo dei valori del tipo di destinazione può presentarsi anche quando un valore intero, con o senza segno, viene convertito in un tipo intero *con segno*, ad esempio da `long` o `unsigned int` a `int`. A differenza delle conversioni verso tipi senza segno, il risultato di un overflow in conversione verso un tipo con segno è lasciato all'implementazione. La maggior parte dei compilatori scarta i bit più significativi della rappresentazione binaria del valore originale e interpreta i bit più bassi secondo il nuovo tipo.

Esempio.

```
#include <stdio.h>
#include <limits.h>
int main() {
    long long int a = (LLONG_MAX - UINT_MAX) + 1;
    int b = a;

    printf("b: %d\n", b);
    printf("a: %lld\n", a);
}
```

```
b: 1
a: 9223372032559808513
```

In esadecimale `LLONG_MAX` è `0x7FFFFFFFFFFFFFFF` e `UINT_MAX` è `0xFFFFFFFF`, quindi

$$a = \texttt{0x7FFFFFFF00000001}.$$

Convertendo in `int` (32 bit) si tengono solo i 32 bit più bassi, `00000001`: perciò $b = 1$.

10.5 Conversioni verso tipi in virgola mobile

Non tutti i valori interi sono rappresentabili esattamente nei tipi in virgola mobile. Ad esempio, benché l'intervallo di valori del tipo `float` includa quello di `long` e `long long`, `float` ha una precisione di sole sei cifre decimali. Quindi alcuni valori `long` non possono essere memorizzati esattamente in un `float`: il risultato della conversione è il valore rappresentabile immediatamente inferiore o superiore.

Esempio.

```
1 float r_var = 16777217;
2 double l_var = 16777217;
3 printf("The rounding error (l_var - r_var) is %.2f\n", l_var - r_var);
4 printf("r_var is %.2f\n", r_var);
```

```
The rounding error (l_var - r_var) is 1.00
r_var is 16777216.00
```

Ogni valore di un tipo in virgola mobile si può rappresentare esattamente in un altro tipo in virgola mobile di precisione maggiore. Quindi, quando un `double` viene convertito in `long double`, o un `float` in `double` o `long double`, il valore è esattamente preservato.

Nelle conversioni da un tipo più preciso a uno meno preciso, invece, il valore da convertire può essere fuori dall'intervallo del nuovo tipo.

- Se il valore supera l'intervallo del tipo di destinazione, il risultato della conversione è *indefinito*.
- Se il valore è nell'intervallo ma non è rappresentabile esattamente con la precisione del tipo di destinazione, il risultato è il valore rappresentabile immediatamente inferiore o superiore.

Riferimenti

Niente sul libro. Risorse indicate dal docente:

- `https://www.w3schools.com/c/c_type_conversion.php`
- `https://www.guru99.com/c-type-casting.html`
- soprattutto, il capitolo 4 di *C in a Nutshell* (O'Reilly): `https://www.oreilly.com/library/view/c-in-a/0596006977/ch04.html`

11 Espressioni, lvalue e operatori

Lezione 11 – 19 ottobre 2026

11.1 Espressioni

Definizione. Un'*espressione* è una sequenza di costanti (literal), identificatori e operatori che il programma valuta eseguendo le operazioni indicate. Lo scopo di un'espressione in un programma può essere ottenere il valore risultante, produrre *effetti collaterali* della valutazione, oppure entrambi.

Una singola costante, un literal stringa o l'identificatore di un oggetto o di una funzione è già di per sé un'espressione.

11.1.1 Effetti collaterali

Un'espressione ha un *effetto collaterale* (*side effect*) se modifica uno stato al di fuori del proprio ambiente locale, cioè se ha un'interazione osservabile con il mondo esterno oltre a restituire un valore.

```
1 if (a == 5) {              // restituisce un valore
2     printf("Hello World!\n");
3 }
4 if (a = 5) {               // restituisce un valore E ha un effetto collaterale
      su a
5     printf("Hello World!\n");
6 }
```

Osservazione. Il secondo `if` è *sbagliato* come modo di verificare una condizione: `a = 5` assegna 5 ad `a` e vale 5, quindi la condizione è sempre vera.

11.1.2 Tipo di un'espressione

Ogni espressione ha un tipo: è il tipo del valore che risulta dalla sua valutazione. Se l'espressione non produce alcun valore, ha tipo `void`. Per esempio, se `a` ha tipo `int`, anche `a + 1` ha tipo `int`.

Per capire come vengono valutate le espressioni in C servono quattro concetti fondamentali: *precedenza* e *associatività* degli operatori, *sequence point* e *lvalue/rvalue*.

11.2 Lvalue e rvalue

Definizione. Un *lvalue* è un'espressione che designa un oggetto. Per *oggetto* si intende una locazione di memoria il cui contenuto può rappresentare valori.

L'esempio più semplice di lvalue è il nome di una variabile. La «l» iniziale significava originariamente *left*: poiché un lvalue designa un oggetto, può comparire a sinistra di un operatore di assegnamento (ma anche a destra), come in

```
espressioneSinistra = espressioneDestra
```

Le espressioni che rappresentano un valore senza designare un oggetto si chiamano, per analogia, *rvalue*. Un rvalue può comparire a destra di un operatore di assegnamento, ma non a sinistra.

Esempio.

Espressione	Categoria	Uso
`7`	rvalue	`7 = a` è un errore; `a = 7` è corretto
`a`	lvalue	`a = 7`
`a+1`	rvalue	`a + 1 = 8` è un errore; `a = a + 1` è corretto
`'c'`	rvalue	`'c' = a` è un errore; `a = 'c'` è corretto
`a+b`	rvalue	`a + b = 11` è un errore; `a = a + b` è corretto

11.2.1 Gli lvalue possono stare anche a destra di =

In `a = 7; b = a;`

1. quando una variabile è a sinistra di `=`, ne *modifichiamo* il valore con quello dell'espressione a destra;
2. quando una variabile è a destra di `=`, ne prendiamo il valore memorizzato.

11.2.2 Variabili

Le variabili sono lvalue: sono contenitori di valori, qualunque cosa il cui valore possa cambiare in memoria. Per esempio, con `int a = 7;` e `int b = 256;` (little endian, 4 byte ciascuna) la variabile `a` occupa i byte a partire dall'indirizzo 2700 con contenuto `07 00 00 00`, e `b` quelli a partire da 2704 con contenuto `00 01 00 00`.

11.2.3 Costanti e lvalue

Un oggetto può essere dichiarato `const`. In tal caso, anche se è un lvalue, non si può usare a sinistra di un assegnamento.

```
1 int a = 1;
2 const int b = 2;
3 b = 20;      // Errore: b è dichiarata const int
```

L'operando sinistro di un assegnamento deve essere un *lvalue modificabile*.

Definizione. Un *lvalue modificabile* è un lvalue che non è dichiarato con il qualificatore `const`.

Si usano le costanti quando si vuole essere sicuri che il valore non cambi durante l'esecuzione del programma. Un valore va assegnato subito, nella definizione, perché dopo non è più modificabile:

```
1 const double pi;         // sbagliato: nessun valore iniziale
2 const double pi = 3.14;  // corretto
3 pi = 3.14;               // sbagliato: pi non è modificabile
```

11.3 Precedenza e associatività degli operatori

11.3.1 Precedenza

Un'espressione può contenere più operatori. In questo caso la *precedenza* degli operatori determina quale parte dell'espressione è l'operando di ciascun operatore. Come nelle regole usuali dell'aritmetica, gli operatori `*`, `/` e `%` hanno precedenza maggiore di `+` e `-`. Per esempio

`a - b * c` equivale a `a - (b * c)`.

Se si vogliono raggruppare gli operandi in modo diverso si usano le parentesi: `(a - b) * c`.

11.3.2 Associatività

Se due operatori in un'espressione hanno la stessa precedenza, la loro *associatività* stabilisce se vengono raggruppati con gli operandi da sinistra a destra o da destra a sinistra. Gli operatori aritmetici associano da sinistra a destra, gli operatori di assegnamento da destra a sinistra:

- `a / b / c` equivale a `(a / b) / c`;
- `a = b = c` equivale a `a = (b = c)`.

11.3.3 Tabella di precedenza e associatività

Prec.	Operatori	Associatività
1	Postfissi: `[]` `()` `.` `->` `++` `--` (e `(tipo){lista }`)	da sinistra a destra
2	Unari: `++` `--` `!` `~` `+` `-` `*` `&` `sizeof`	da destra a sinistra
3	Cast: `(tipo)`	da destra a sinistra
4	Moltiplicativi: `*` `/` `%`	da sinistra a destra
5	Additivi: `+` `-`	da sinistra a destra
6	Shift: `<<` `>>`	da sinistra a destra
7	Relazionali: `<` `<=` `>` `>=`	da sinistra a destra
8	Uguaglianza: `==` `!=`	da sinistra a destra
9	AND bit a bit: `&`	da sinistra a destra
10	XOR bit a bit: `^`	da sinistra a destra
11	OR bit a bit: `|`	da sinistra a destra
12	AND logico: `&&`	da sinistra a destra
13	OR logico: `||`	da sinistra a destra
14	Condizionale: `? :`	da destra a sinistra
15	Assegnamento: `= += -= *= /= %= &= ^= |= <<= >>=`	da destra a sinistra
16	Virgola: `,`	da sinistra a destra

Alcuni simboli compaiono due volte nella tabella.

- Gli operatori di incremento e decremento `++` e `--` hanno precedenza maggiore quando sono usati come operatori *postfissi* (come in `x++`) che come operatori *prefissi* (come in `++x`).
- I simboli `+`, `-`, `*` e `&` rappresentano sia operatori *unari*, cioè che lavorano su un solo operando, sia operatori *binari*, che collegano due operandi.

11.4 Operatori aritmetici

Operatore	Significato	Esempio	Risultato
`*`	moltiplicazione	`x * y`	prodotto di x e y
`/`	divisione	`x / y`	quoziente di x diviso y
`%`	modulo	`x % y`	resto di x diviso y
`+`	addizione	`x + y`	somma di x e y
`-`	sottrazione	`x - y`	differenza di x e y
`+` (unario)	segno positivo	`+x`	il valore di x
`-` (unario)	segno negativo	`-x`	l'opposto aritmetico di x

Gli operandi degli operatori aritmetici sottostanno alle regole seguenti:

- solo l'operatore `%` richiede operandi *interi*; gli operandi di tutti gli altri operatori possono avere qualunque tipo aritmetico;

- gli operandi sono soggetti alle usuali conversioni aritmetiche (lezione 10). Il risultato di una divisione tra due operandi interi è anch'esso un intero.

Esempio. Con `short n = -5;`

Espressione	Conversione implicita	Tipo	Valore
`-n`	promozione intera	`int`	5
`n * -2L`	n promosso a `long` (la costante `-2L` è `long`)	`long`	10
`8 / n`	promozione intera	`int`	-1
`8 % n`	promozione intera	`int`	3
`8.0 / n`	n convertito a `double` (perché 8.0 è `double`)	`double`	$-1{,}6$
`8.0 % n`	errore: `%` richiede operandi interi	—	—

Osservazione. Se entrambi gli operandi di una moltiplicazione o di una divisione hanno lo stesso segno, il risultato è positivo; altrimenti è negativo. Il risultato dell'operazione di modulo, invece, ha sempre lo stesso segno dell'operando sinistro (come si vede sopra: $8\%(-5) = 3$).

11.5 Operatori di assegnamento

In un'operazione di assegnamento l'operando sinistro deve essere un lvalue modificabile, cioè un'espressione che designa un oggetto il cui valore può essere cambiato. Nell'*assegnamento semplice*, eseguito con l'operatore `=`, il valore dell'operando destro viene memorizzato in questo oggetto.

Esistono anche gli *assegnamenti composti*, che combinano nello stesso passo un'operazione aritmetica o bit a bit con l'assegnamento:

- `*=` `/=` `%=` `+=` `-=` (operazione aritmetica e assegnamento);
- `<<=` `>>=` `&=` `^=` `|=` (operazione bit a bit e assegnamento).

Nel valutare un assegnamento composto il programma combina i due operandi con l'operazione indicata e assegna il risultato all'operando sinistro: `a += b` equivale a `a = a + b`.

11.6 Incremento e decremento

Ciascuno dei simboli `++` e `--` rappresenta sia un operatore postfisso sia un operatore prefisso.

Operatore	Effetto collaterale	Valore dell'espressione
`x++` (postfisso)	aumenta x di uno (come `x = x + 1`)	il valore che x aveva *prima* dell'incremento
`++x` (prefisso)	aumenta x di uno	il valore che x ha *dopo* l'incremento
`x--` (postfisso)	diminuisce x di uno (come `x = x - 1`)	il valore che x aveva *prima* del decremento
`--x` (prefisso)	diminuisce x di uno	il valore che x ha *dopo* il decremento

11.7 Operatori di confronto

Gli operatori di confronto, detti anche *relazionali* e di *uguaglianza*, confrontano due operandi e producono un valore di tipo `int`: 1 se la relazione specificata è vera, 0 se è falsa.

Esempio.

```
int a = 5;
int b = 6;
int c = 0;
c = a < b;      // c vale 1
c = a == b;     // c vale 0
c = b >= a;     // c vale 1
```

Gli operatori di confronto hanno precedenza minore degli operatori aritmetici, ma maggiore degli operatori logici. Inoltre gli operatori di uguaglianza `==` e `!=` hanno precedenza minore degli altri operatori di confronto:

- `a < b && b < c + 1` equivale a `(a < b) && (b < (c + 1))`;
- `a < b != b < c` equivale a `(a < b) != (b < c)`.

11.8 Operatori logici

Si possono collegare più espressioni con gli operatori logici per formare condizioni composte: le operazioni booleane AND (`&&`), OR (`||`) e NOT (`!`). Come le espressioni di confronto, anche le espressioni logiche hanno tipo `int`: il risultato vale 1 se l'espressione logica è vera e 0 se è falsa.

Qualunque operando con valore 0 viene interpretato come falso; qualunque valore diverso da 0 è trattato come vero. Le condizioni seguenti sono tutte equivalenti:

```
(a < -0.2) || (a > 0.2)
a < -0.2 || a > 0.2
!(a >= -0.2 && a <= 0.2)
```

L'operatore unario `!` ha precedenza maggiore degli altri.

11.8.1 Valutazione in corto circuito

Gli operatori `&&` e `||` hanno una particolarità importante: i loro operandi vengono valutati da sinistra a destra e, se il valore dell'operando sinistro è sufficiente a determinare il risultato, l'operando destro *non viene valutato affatto*.

- `&&` valuta l'operando destro solo se quello sinistro produce un valore diverso da zero;
- `||` valuta l'operando destro solo se quello sinistro produce 0.

Esempio.

```
int a = 1;
int b = 0;
int c;
c = a || ++b;    // b non viene incrementato (resta 0), c vale 1
```

```c
int a = 1;
int b = 0;
int c;
c = b && ++a;     // a non viene incrementato (resta 1), c vale 0
```

12 Operatori bit a bit, altri operatori e sequence point

Lezione 12 – 19 ottobre 2026

12.1 Operatori bit a bit

Gli operatori bit a bit permettono di manipolare i singoli bit di un byte o di un'unità di dati più grande.

Operatore	Significato	Esempio	Risultato (per ogni posizione di bit)
`&`	AND bit a bit	`x & y`	1 se 1 sia in x sia in y; 0 se 0 in x o in y, o in entrambi
`|`	OR bit a bit	`x | y`	1 se 1 in x o in y, o in entrambi; 0 se 0 in entrambi
`^`	XOR bit a bit (OR esclusivo)	`x ^ y`	1 se 1 in x o in y ma non in entrambi; 0 se i due bit sono uguali
`~`	NOT bit a bit (complemento a uno)	`~x`	1 se 0 in x; 0 se 1 in x

Esempio. Con `int a = 6;` e `int b = 11;` (mostrati gli ultimi 5 bit, gli altri valgono 0 tranne che in $\sim a$, dove valgono 1):

Espressione	Schema di bit
`a = 6`	`00110`
`b = 11`	`01011`
`a & b`	`00010`
`a | b`	`01111`
`a ^ b`	`01101`
`~a`	`11001`

Un uso tipico degli assegnamenti composti bit a bit è la *maschera*:

```c
a &= 0xFF;     // equivale a a = a & 0xFF; azzera tutti i bit tranne gli otto
    meno significativi
```

12.1.1 Rappresentazione in memoria e nel processore

In memoria i valori sono rappresentati in *little endian*; nel processore gli stessi valori vengono tradotti in *big endian*. Tutte le operazioni nel processore sono quindi eseguite in big endian e, quando il risultato viene riscritto in memoria, viene salvato in little endian.

12.2 Operatori di shift

Gli operatori di shift traslano lo schema di bit dell'operando sinistro del numero di posizioni indicato dall'operando destro.

Gli operandi degli operatori di shift devono essere interi. Prima dello shift vengono eseguite su entrambi le promozioni intere. Il valore dell'operando destro non deve essere negativo e deve essere minore della larghezza (in bit) dell'operando sinistro dopo la promozione intera.

Esempio.

```
1 unsigned long n = 0xB,   // schema di bit: 0 ... 0 0 0 1 0 1 1
2               result = 0;
3 result = n << 2;         // 0 ... 0 1 0 1 1 0 0
4 result = n >> 2;         // 0 ... 0 0 0 0 0 1 0
```

Osservazione. Uno shift a sinistra di y posizioni equivale a moltiplicare l'operando sinistro per 2^y; uno shift a destra di y posizioni equivale a dividere per 2^y (divisione intera): $x/2^y$.

12.3 Altri operatori

In C ci sono altri sei operatori che non rientrano nelle categorie viste.

Operatore	Significato	Esempio	Risultato
`()`	chiamata di funzione	`log(x)`	passa il controllo alla funzione specificata, con gli argomenti specificati
`sizeof`	dimensione in byte di un oggetto o di un tipo	`sizeof x`	numero di byte occupati da x in memoria
`(tipo)`	conversione esplicita di tipo (cast)	`(short) x`	il valore di x convertito al tipo specificato
`?:`	valutazione condizionale	`x ? y : z`	il valore di y se x è vero (diverso da 0), altrimenti il valore di z
`,`	valutazione sequenziale	`x, y`	valuta prima x, poi y; il risultato è il valore di y

(gli altri due operatori, di selezione dei membri e di indicizzazione, sono già nella tabella di precedenza.)

12.3.1 Operatore condizionale

L'operatore condizionale è detto anche operatore *ternario*, perché è l'unico che ha tre operandi:

```
condizione ? espressione1 : espressione2
```

L'operazione valuta prima la condizione. Poi, a seconda del risultato, valuta una delle due espressioni alternative. Dopo la valutazione della condizione c'è un *sequence point*. Se il risultato è diverso da 0 (la condizione è vera) viene valutato *solo* il secondo operando, `espressione1`, e l'intera operazione assume il suo valore. Se invece la condizione è 0 (falsa) viene valutato solo il terzo operando, `espressione2`, e l'operazione assume il suo valore.

Esempio.

```
int a = 6;
int b = 12;
int c;
c = b > a ? 100 : 150;      // c diventa 100
c = a == b ? 100 : 150;     // c diventa 150
```

Il primo operando, la condizione, deve avere tipo aritmetico. Se entrambe le espressioni alternative hanno tipo aritmetico, il tipo del risultato è quello che si ottiene applicando agli operandi le usuali conversioni aritmetiche:

```
int a = 6;
int b = 12;
int c;
c = (b > a) ? 100 : 150LL;   // l'intera espressione ha tipo long long
```

12.3.2 Operatore virgola

L'operatore virgola è un operatore binario:

```
espressione1 , espressione2
```

Garantisce un'elaborazione sequenziale: prima viene valutato l'operando sinistro, poi quello destro. Il risultato dell'intera espressione ha il tipo e il valore dell'operando destro. L'operando sinistro viene valutato solo per i suoi effetti collaterali; il suo valore viene scartato.

Esempio.

```
int a = 6;
int b = 12;
int c;
c = (a++, a + 7);   // a diventa 7, c diventa 14
```

Il valore dell'intera espressione tra parentesi è 14.

12.4 Sequence point ed effetti collaterali

Una funzione o un'espressione ha un *effetto collaterale* se, oltre a restituire un valore, modifica uno stato o ha un'interazione osservabile con le funzioni chiamanti o con il mondo esterno.

Definizione. Un *sequence point* è un punto dell'esecuzione di un programma in cui è garantito che

- tutti gli effetti collaterali delle valutazioni precedenti sono già stati eseguiti, e
- nessun effetto collaterale delle valutazioni successive è ancora stato eseguito.

In altre parole, è un punto in cui tutti gli effetti collaterali sono stati valutati prima di passare al passo successivo.

12.4.1 Attenzione agli assegnamenti

La modifica dell'operando sinistro è un effetto collaterale di un'espressione di assegnamento. Il valore dell'intera espressione di assegnamento è uguale al valore assegnato all'operando sinistro, e l'espressione ha il tipo dell'operando sinistro. A differenza dell'operando sinistro, però, l'espressione di assegnamento *non è un lvalue*.

Esempio.

```
int a = 6;
int b = 12;
int c;
c = (b = a);
```

L'espressione `(b = a)` ha valore 6 e, come effetto collaterale, cambia il valore di `b` in memoria; poi l'assegnamento a `c` ha a sua volta come effetto collaterale la modifica di `c`.

Esempio.

```
int a = 6;
int b = 12;
int c;
(c = b) = a;     // errore di compilazione
```

Il compilatore segnala un errore perché `(c = b)` è un rvalue e non un lvalue: non si può assegnargli il valore di `a`.

Esempio (Quanti effetti collaterali?)**.**

```
int a = 3;
int b = 4;
a = b++;
```

L'espressione `a = b++` ha due effetti collaterali: uno su `b` e uno su `a`.

12.4.2 Regola da ricordare

Un programmatore deve ricordarsi di *non modificare la stessa variabile più di una volta tra due sequence point consecutivi.*

```
i = 1;       // OK
i = i++;     // SBAGLIATO: due modifiche di i; il comportamento è indefinito
```

Poiché l'assegnamento e l'incremento nell'ultima istruzione possono avvenire in un ordine qualunque, il valore risultante di `i` è indefinito.

```
int i = i++;   // OK, qualunque cosa significhi: è la creazione di i
               // seguita dalla sua inizializzazione; il primo "=" non è
```

```
3                   // un operatore di assegnamento
```

12.4.3 Dove si trovano i sequence point

I sequence point più importanti si trovano nelle posizioni seguenti:

- alla fine di un comando (`;`);
- dopo la valutazione di tutti gli argomenti di una chiamata di funzione e prima che il controllo passi alle istruzioni della funzione;
- alla fine di un'espressione che non fa parte di un'espressione più grande: per esempio ognuna delle tre espressioni di controllo di un `for`, la condizione di un `if` o di un `while`, l'espressione di un `return` e gli inizializzatori (dopo la valutazione di 5 in `int a = 5;`);
- dopo la valutazione del primo operando di ciascuno dei seguenti operatori: `&&` (AND logico), `||` (OR logico), `?:` (condizionale) e `,` (virgola).

Esempio.

- L'espressione `++i < 100 ? f(i++) : (i = 0)` è ammessa, perché c'è un sequence point tra la prima modifica di `i` e quella delle altre due che viene eseguita.
- Anche `a++ && a++` è corretta (sequence point dopo il primo operando di `&&`).
- `a++ + b++` è corretta, perché ha un effetto collaterale su `a` e uno su `b`, cioè su due variabili diverse.

Esercizio. Che cosa stampa il programma seguente?

```c
int main()
{
    int i = 0;
    int k = i++, j = i++;
    printf("%d %d %d\n", k, j, i);
    int l = 0;
    int m = ++l, n = ++l;
    printf("%d %d %d\n", m, n, l);
    int q;
    q = ++q;
    printf("%d\n", q);
}
```

```
0 1 2
1 2 2
```

Nelle prime due dichiarazioni c'è un sequence point alla fine di ciascun inizializzatore, quindi i valori sono ben definiti. L'ultima istruzione, invece, modifica `q` due volte senza sequence point intermedio: il compilatore avverte con `warning: multiple unsequenced modifications to 'q' [-Wunsequenced]` e il comportamento è indefinito.

12.5 Riferimenti

Sul libro: sezioni 2.5, 2.6, 3.11, 3.12, 4.9, 4.10, 7.7, 10.9, appendice A, pag. 77 (espressione condizionale), pag. 87 (cast), pag. 135 (lvalue e rvalue), pag. 243 (virgola). Per effetti collaterali e sequence point:

- `https://learn.microsoft.com/it-it/cpp/c-language/side-effects?view=msvc-170`
- `https://learn.microsoft.com/it-it/cpp/c-language/c-sequence-points?view=msvc-170`

13 Istruzioni, blocchi e cicli

Lezione 13 – 26 ottobre 2026

13.1 Istruzioni

Un'*istruzione* (*statement*) specifica una o più azioni da eseguire, come assegnare un valore a una variabile, passare il controllo a una funzione o saltare a un'altra istruzione. L'insieme di tutte le istruzioni di un programma determina ciò che il programma fa.

I salti e i cicli sono istruzioni che controllano il flusso del programma. Eccetto quando queste istruzioni di controllo producono dei salti, le istruzioni vengono eseguite in sequenza, cioè nell'ordine in cui compaiono nel programma. Un'istruzione semplice è un'espressione seguita da un punto e virgola:

```
[espressione] ;
```

Esempio. `a + 3` è un'*espressione*; `a + 3;` è un *comando* (istruzione). Si veda la differenza:

```c
int a = 3;
a = a + 2;        // istruzione: espressione seguita da ;
if (a + 3;) { }   // SBAGLIATO: dentro le parentesi dell'if serve un'espressione
    , non un'istruzione
```

13.1.1 Programmazione imperativa e procedurale

In informatica la *programmazione imperativa* è un paradigma di programmazione che usa istruzioni che cambiano lo stato del programma. La *programmazione procedurale* è un paradigma derivato dalla programmazione strutturata, basato sul concetto di chiamata di procedura. Le procedure sono dette anche routine, sottoprogrammi o funzioni. Il C è un linguaggio imperativo e procedurale.

13.1.2 Altri paradigmi

La *programmazione dichiarativa* è un paradigma che esprime la logica di una computazione senza descriverne il flusso di controllo: descrivendo *che cosa* il programma deve ottenere, piuttosto che *come* ottenerlo, si cerca di minimizzare o eliminare gli effetti collaterali. Un esempio in Prolog:

```
mother_child(trude, sally).
father_child(tom, sally).
father_child(tom, erica).
father_child(mike, tom).
sibling(X, Y) :- parent_child(Z, X), parent_child(Z, Y).
parent_child(X, Y) :- father_child(X, Y).
```

```
parent_child(X, Y) :- mother_child(X, Y).

?- sibling(sally, erica).
Yes
```

13.2 Istruzione composta (blocco)

Un'*istruzione composta*, detta in breve *blocco*, raggruppa una serie di istruzioni e dichiarazioni tra parentesi graffe in modo da formare un'unica istruzione:

{ [lista di dichiarazioni e istruzioni] }

A differenza delle istruzioni semplici, i blocchi *non* sono terminati da un punto e virgola.

Un blocco si usa dove la sintassi richiede una sola istruzione, ma lo scopo del programma ne richiede più di una.

```
{
    int a = 5, b = 4, c = 0;
    c = a + b;
}
```

Esempio. Senza le graffe l'`if` controlla soltanto la prima istruzione che lo segue:

```
int a = -4;
if (a < 0)
    printf("%d is less than 0", a);
    a = -a;
    printf("a now is %d", a);
```

```
-4 is less than 0 a now is 4
```

Con `int a = 4;` nello stesso codice `a = -a;` e la seconda `printf` vengono comunque eseguite, perché non sono sotto il controllo dell'`if`; l'output è `a now is -4`. Per controllare tutte e tre le istruzioni occorre un blocco:

```
int a = 4;
if (a < 0) {
    printf("%d is less than 0", a);
    a = -a;
    printf("a now is %d", a);
}
```

13.2.1 Blocchi e scope

```
{   double result = 0.0, x = 0.0;
    long status = 0;
    int limit;
    ++x;                    // istruzioni
    if (status == 0)
    {                       // nuovo blocco
        int i = 0;
        { int k = 3; }      // un altro blocco
    }
}
```

I nomi dichiarati all'interno di un blocco hanno *block scope*: sono visibili solo dalla loro dichiarazione fino alla fine del blocco. All'interno di questo scope, una dichiarazione può anche nascondere un oggetto con lo stesso nome dichiarato fuori dal blocco.

13.3 Istruzioni di definizione

Servono a definire variabili. La definizione di una variabile equivale alla sua creazione in memoria. Nello stesso scope le variabili devono avere nomi diversi.

```
int main() {
    int a = 4;      // corretto
}
```

```
int main() {
    int a = 4;
    float a = 5;    // errore: a è già definita in questo scope
}
```

Non si possono usare come espressioni:

```
int main() {
    int a = int b = 3;    // errore
}
```

13.4 Cicli

Si usa un *ciclo* per eseguire un gruppo di istruzioni, detto *corpo del ciclo*, più di una volta. In C un ciclo si introduce con una delle tre istruzioni di iterazione:

1. `while`;
2. `do ... while`;
3. `for`.

Il numero di iterazioni del corpo è controllato da una condizione, l'*espressione di controllo*: un'espressione di tipo scalare, cioè un'espressione aritmetica o un puntatore. La condizione del ciclo è vera se il valore dell'espressione di controllo è diverso da 0, altrimenti è considerata falsa.

13.4.1 Istruzione while

Un'istruzione `while` esegue un'istruzione ripetutamente finché l'espressione di controllo è vera:

```
while ( espressione ) istruzione
```

Il `while` è un ciclo *a controllo in testa* (*top-driven*): prima si valuta la condizione del ciclo (l'espressione di controllo). Se è vera, si esegue il corpo e si valuta di nuovo l'espressione di controllo. Se la condizione è falsa, l'esecuzione prosegue con l'istruzione che segue il corpo del ciclo.

Sintatticamente il corpo del ciclo è *una* istruzione; se ne servono più d'una si raggruppano in un blocco.

Esempio.

```c
#include <stdio.h>
int main() {
    int a = 10;

    while (a < 20) {
        printf("value of a: %d\n", a);
        a++;
    }
    return 0;
}
```

```
value of a: 10
value of a: 11
value of a: 12
value of a: 13
value of a: 14
value of a: 15
value of a: 16
value of a: 17
value of a: 18
value of a: 19
```

13.4.2 Istruzione for

Come il `while`, l'istruzione `for` è un ciclo a controllo in testa, ma con più logica del ciclo contenuta nell'istruzione stessa:

```
for ( [espressione1]; [espressione2]; [espressione3] ) istruzione
```

- `espressione1`: *inizializzazione*. Viene valutata una sola volta, prima della prima valutazione dell'espressione di controllo, per eseguire le inizializzazioni necessarie.
- `espressione2`: *espressione di controllo*. Viene verificata prima di ogni iterazione; l'esecuzione del ciclo termina quando vale falso.
- `espressione3`: *aggiustamento*, per esempio l'incremento di un contatore, eseguito dopo ogni iterazione del ciclo e prima che l'espressione di controllo venga verificata di nuovo.

Esempio.

```
int a = 0;
int i;
for (i = 0; i < 4; ++i) {
    a = a + 2;
    printf("%d", a);
}
```

```
2468
```

La variabile `i` vale successivamente $0, 1, 2, 3$ (e 4, quando il ciclo termina). Lo stesso comportamento si ottiene con `for (i = 1; i <= 4; ++i)`.

Ognuna delle tre espressioni nell'intestazione del `for` può essere omessa; la forma più breve possibile è quindi `for (;;)`. Un'espressione di controllo mancante è considerata sempre vera e definisce quindi un ciclo infinito. `for (; espressione; )` equivale a `while (espressione)`. Ogni `for` si può riscrivere come `while` e viceversa:

```
int a = 0, i = 0;
while (i < 4)
{
    a = a + 2;
    ++i;
}
```

Dichiarazione al posto di espressione1

Dallo standard C99 si può usare una dichiarazione al posto di `espressione1`. In tal caso lo scope della variabile dichiarata è limitato al ciclo `for`:

```
for (int i = 0; i < 4; ++i) {
    a = a + 2;
}
```

La variabile `i` dichiarata in questo ciclo non esiste più dopo la fine del ciclo.

13.4.3 Istruzione do ... while

L'istruzione `do ... while` è un ciclo *a controllo in coda* (*bottom-driven*):

```
do istruzione while ( espressione );
```

Il corpo del ciclo viene eseguito una volta prima che l'espressione di controllo sia valutata per la prima volta. A differenza di `while` e `for`, il `do ... while` garantisce che venga eseguita *almeno una* iterazione del corpo. Se l'espressione di controllo è vera segue un'altra iterazione; se è falsa il ciclo termina.

Esempio.

```c
#include <stdio.h>
int main()
{
    double number, sum = 0;

    do   // il corpo viene eseguito almeno una volta
    {
        printf("Enter a number: ");
        scanf("%lf", &number);
        sum += number;
    }
    while (number != 0.0);

    printf("Sum = %.2lf", sum);
    return 0;
}
```

```
Enter a number: 1.5
Enter a number: 2.4
Enter a number: -3.4
Enter a number: 4.2
Enter a number: 0
Sum = 4.70
```

13.5 Riferimenti

Sul libro: scope o campo di azione, sezione 5.13; sezioni 3.4–3.10, 4.1–4.8 (cicli e istruzioni); sezione 15.9 (da poi scordare immediatamente).

14 Selezione, salti e typedef

Lezione 14 – 26 ottobre 2026

14.1 Istruzioni di selezione

Un'istruzione di selezione può dirigere il flusso di esecuzione del programma lungo percorsi diversi a seconda di una condizione data. In C ci sono due istruzioni di selezione:

1. `if`;
2. `switch`.

14.1.1 Istruzione if

Un'istruzione `if` ha la forma

```
if ( espressione ) istruzione1 [ else istruzione2 ]
```

La clausola `else` è facoltativa. L'espressione viene valutata per prima, per determinare quale delle due istruzioni viene eseguita; deve avere tipo scalare. Se il suo valore è vero (cioè diverso da 0) viene eseguita `istruzione1`; altrimenti viene eseguita `istruzione2`, se presente.

```
if (testExpression)
{
    // corpo dell'if
}
// istruzione subito dopo l'if
```

```
if (testExpression)
{
    // corpo dell'if
}
else
{
    // corpo dell'else
}
// istruzione subito dopo l'if...else
```

If annidati

Se più istruzioni `if` sono annidate, una clausola `else` appartiene sempre all'ultimo `if` (allo stesso livello di annidamento dei blocchi) che non ha ancora un `else`:

```
if (n > 0)
    if (n % 2 == 0)
        puts("n is positive and even");
    else    // alternativa all'ultimo if
        puts("n is positive and odd");
```

Una clausola `else` può essere assegnata a un `if` diverso racchiudendo in un blocco l'ultimo `if` che non deve avere un `else`:

```
if (n > 0)
{
    if (n % 2 == 0)
        puts("n is positive and even");
}
else        // alternativa al primo if
    puts("n is negative or zero");
```

Catena else if

Per scegliere una tra più di due istruzioni alternative si possono mettere a cascata degli `if` in una catena `else if`. Ogni nuovo `if` è semplicemente annidato nella clausola `else` di quello precedente:

```
double spec = 10.0, measured = 10.7, diff;
diff = measured - spec;
if (diff >= 0.0 && diff < 0.5)
    printf("Rounding down: %.2f\n", diff);
else if (diff >= 0.5 && diff < 1)
```

```
6       printf("Rounding up: %.2f\n", diff);
7   else
8       printf("Difference greater than 1!\n");
```

Appena una di queste espressioni è vera, viene eseguita l'istruzione corrispondente e le altre vengono scartate. Se nessuna delle condizioni è vera, viene eseguita la clausola `else` dell'ultimo `if`, se presente.

Osservazione. Nelle slide la seconda `printf` stampava ancora «Rounding down»: qui è stata corretta in «Rounding up», coerentemente con l'intervallo [0,5, 1).

14.1.2 Istruzione switch

Un'istruzione `switch` fa saltare il flusso di esecuzione a una tra più istruzioni, a seconda del valore di un'espressione intera:

`switch ( espressione ) istruzione`

`espressione` ha tipo intero e `istruzione` è il *corpo dello switch*, che contiene etichette `case` e al più un'etichetta `default`. L'espressione viene valutata una sola volta e confrontata con le espressioni costanti nelle etichette `case`. Se il valore coincide con una delle costanti, il flusso del programma salta all'istruzione che segue quell'etichetta. Se nessuna costante coincide, il programma continua all'etichetta `default`, se c'è.

```
1  switch (menu())   // salta in base al risultato di menu()
2  {
3      case 'A': action1();     // esegue l'azione 1
4                break;         // non esegue nessun'altra "azione"
5      case 'B': action2();     // esegue l'azione 2
6                break;         // non esegue l'azione di default
7      default:  putchar('\a'); // nessun comando riconosciuto
8  }
```

Intervalli

Per trattare un intervallo di valori con la stessa azione, si possono scrivere più etichette consecutive:

```
1  switch (value)
2  {
3      case 1:
4      case 2:
5      case 3:
6      case 4:
7      case 5:
8      case 6:
9      case 7:
10     case 8:
11         printf("Hello, 1 to 8\n");
12         break;
13     default:
14         printf("Hello, default\n");
```

```
        break;
}
```

Esiste anche un'estensione di GCC, non standard, che permette di scrivere `case 1 ... 8:`. Un'alternativa è mettere nel `default` un `if`:

```
switch (value) {
    case 1:
        /* do 1 */
        break;
    case 2:
        /* do 2 */
        break;
    default:
        if (value >= 3 && value <= 8)
            printf("Hello, 3 to 8\n");
}
```

Se però si deve lavorare con molti intervalli, conviene usare gli `if`.

Senza break: fall-through

Senza `break` l'esecuzione prosegue nelle etichette successive.

Esempio.

```
#include <stdio.h>
int main()
{
    int Grade = 'A';
    switch (Grade)
    {
        case 'A': printf("Excellent\n");
        case 'B': printf("Good\n");
        case 'C': printf("OK\n");
        case 'D': printf("Mmmmm....\n");
        case 'F': printf("You must do better than this\n");
        default : printf("What is your grade anyway?\n");
    }
}
```

```
Excellent
Good
OK
Mmmmm....
You must do better than this
What is your grade anyway?
```

14.2 Salti incondizionati

Le istruzioni di salto interrompono l'esecuzione sequenziale delle istruzioni, in modo che l'esecuzione continui in un altro punto del programma. Un salto distrugge le variabili automatiche se la destinazione del salto è fuori dal loro scope. In C ci sono quattro istruzioni che causano salti incondizionati: `break`, `continue`, `goto` e `return`.

14.2.1 break

L'istruzione `break;` può comparire solo nel corpo di un ciclo o di uno `switch` e causa un salto alla prima istruzione dopo il ciclo o lo `switch` che la contiene immediatamente. Si può quindi usare `break` per terminare l'esecuzione di un ciclo in qualunque punto del corpo.

Esempio.

```c
#include <stdio.h>
int main()
{
    int i;
    double number, sum = 0.0;

    for (i = 1; i <= 10; ++i)
    {
        printf("Enter a n%d: ", i);
        scanf("%lf", &number);

        // se l'utente inserisce un numero negativo, il ciclo termina
        if (number < 0.0)
        {
            break;
        }
        sum += number;
    }
    printf("Sum = %.2lf", sum);
    return 0;
}
```

Con input 2,4, 4,5, 3,4, -3 l'output è `Sum = 10.30`.

14.2.2 continue

L'istruzione `continue;` si può usare solo nel corpo di un ciclo e fa saltare il resto dell'iterazione corrente.

- In un ciclo `while` o `do ... while` il programma salta alla successiva valutazione dell'espressione di controllo.
- In un ciclo `for` il programma salta alla successiva valutazione della terza espressione dell'istruzione `for`, che contiene le operazioni eseguite dopo ogni iterazione.

Esempio. Stesso programma di prima, ma con `continue` al posto di `break`:

```c
for (i = 1; i <= 10; ++i)
{
```

```
3       printf("Enter a n%d: ", i);
4       scanf("%lf", &number);
5
6       // se l'utente inserisce un numero negativo, si passa all'iterazione
        successiva
7       if (number < 0.0)
8       {
9           continue;
10      }
11      sum += number;
12  }
```

Con input 1,1, 2,2, 5,5, 4,4, −3,4, −45,5, 34,5, −4,2, −1000, 12 i negativi vengono saltati e l'output è `Sum = 59.70`.

14.2.3 goto

L'istruzione `goto` causa un salto incondizionato a un'altra istruzione nella stessa funzione. La destinazione del salto è specificata dal nome di un'*etichetta*:

```
goto nome_etichetta;
```

Un'etichetta è un nome seguito da due punti: `nome_etichetta: istruzione`. Le etichette hanno un proprio spazio dei nomi, quindi possono avere gli stessi nomi di variabili o tipi senza causare conflitti. Le etichette possono essere messe prima di qualunque istruzione, e un'istruzione può avere più etichette. Servono solo come destinazioni dei `goto` e non hanno alcun effetto se l'istruzione etichettata viene raggiunta nel normale corso dell'esecuzione sequenziale.

```
1  void calculate(int a)
2  {
3      if (a < 1 || a > 5)
4          goto here;
5      printf("a between 1 and 5!\n");
6      return;
7  here: printf("a < 1 or a > 5!\n");
8      return;
9  }
```

Osservazione. Poiché il codice che fa largo uso di `goto` è difficile da leggere, conviene usarli solo quando offrono un vantaggio evidente, come l'uscita rapida da cicli profondamente annidati. Qualunque programma C che usa `goto` si può scrivere anche senza.

Lo *spaghetti code* è un'espressione spregiativa per il codice sorgente con una struttura di controllo complessa e ingarbugliata, soprattutto quando usa molti `goto`, eccezioni, thread o altri costrutti di salto «non strutturati» (tipico di assembly, Fortran, BASIC).

14.2.4 return

Si veda la lezione sulle funzioni: `return` è anch'esso un'istruzione di salto incondizionato, come `goto`.

14.3 typedef

Il modo più semplice di usare tipi con nomi complessi è dichiarare dei *sinonimi* semplici per essi. In una dichiarazione che inizia con la parola chiave `typedef`, ogni dichiaratore definisce un identificatore come sinonimo del tipo specificato. L'identificatore è poi detto *typedef name* per quel tipo. A parte la parola chiave `typedef`, la sintassi è esattamente la stessa di una dichiarazione di un oggetto o di una funzione del tipo specificato.

Esempio.

```
1 typedef unsigned int UINT;
2 typedef enum state {DEAD, ALIVE} State;
3
4 UINT ui = 10;
5 State s = DEAD;
```

Nello scope di queste dichiarazioni `UINT` è sinonimo di `unsigned int` e `State` è sinonimo di `enum state`. La variabile `ui` ha tipo `unsigned int` e `s` ha tipo `enum state`.

14.4 Riferimenti

Sul libro: sezione 10.6 (`typedef`).

15 Array, stringhe e matrici

Lezione 15 – 2 novembre 2026

15.1 Array

Un *array* contiene oggetti di un dato tipo, memorizzati consecutivamente in un blocco continuo di memoria. I singoli oggetti sono detti *elementi* dell'array. Il tipo degli elementi può essere qualunque tipo di oggetto.

Il tipo di un array è determinato dal tipo e dal numero degli elementi. Se gli elementi hanno tipo T, l'array è detto «array di T». Se gli elementi hanno tipo `int`, per esempio, il tipo dell'array è «array di `int`». Il tipo è però *incompleto*, a meno che non si specifichi anche il numero di elementi: un array di `int` con 16 elementi ha un tipo di oggetto completo, «array di 16 elementi `int`».

15.1.1 Operatore di indicizzazione

L'operatore di indicizzazione `[]` permette di accedere ai singoli elementi di un array. Ha due operandi. Nel caso più semplice un operando è il nome di un array e l'altro è un intero. L'espressione `myarray[i]` designa allora l'elemento numero i dell'array, dove il primo elemento ha numero zero. L'operando sinistro di `[]` non deve necessariamente essere il nome di un array; l'operando destro deve avere tipo intero.

15.1.2 Definizione di array

È un tipo *aggregato*: una sequenza di oggetti con lo stesso tipo. La definizione di un array determina il suo nome, il tipo dei suoi elementi e il numero degli elementi. Una definizione di array senza inizializzazione esplicita ha la sintassi

```
tipo nome[ numero_di_elementi ];
```

Il numero di elementi, tra parentesi quadre, deve essere un'espressione intera con valore maggiore di zero. Per esempio:

```
1 char buffer[4*512];
```

Questa riga definisce un array di nome `buffer` formato da 2048 elementi di tipo `char`; `sizeof(buffer)` vale $2048 \cdot$ `sizeof(char)`.

15.1.3 Array a lunghezza variabile

Dallo standard C99 è possibile definire un array usando per il numero di elementi un'espressione non costante (*variable-length array*, VLA). Dal C11 sono una caratteristica *opzionale* dei compilatori: alcuni compilatori potrebbero non implementarli.

```
1 void func(int n)
2 {
3     int vla1[2*n];
4     int vla2[n];
5     /* ... */
6 }
```

15.1.4 Accesso agli elementi

L'espressione `myArray[i]` designa l'elemento dell'array con indice i. Gli elementi sono indicizzati a partire da 0: se `len` è il numero di elementi di un array, l'ultimo elemento ha indice `len` $- 1$.

```
1 const int a = 4;
2 long mArray[a];
3 for (int i = 0; i < a; ++i)
4     mArray[i] = 2 * i;
```

mArray[0]	mArray[1]	mArray[2]	mArray[3]
0	2	4	6

15.1.5 Inizializzazione

Per inizializzare esplicitamente un array quando lo si definisce, si usa una *lista di inizializzazione*: una lista di inizializzatori separati da virgole.

```
1 int a[4] = { 1, 2, 4, 8 };   // a[0] = 1, a[1] = 2, a[2] = 4, a[3] = 8
```

Esempio (Accesso fuori dai limiti)**.**

```
1 int a[10] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
2
3 for (int i = 0; i < 20; i++) {
4     printf("Element[%d] = %d\n", i+1, a[i]);
5 }
```

I primi 10 elementi stampati sono $1, \ldots, 10$; dall'undicesimo in poi si leggono valori casuali (per esempio 1339107492, 2097207605, 32767, ...): il ciclo accede a locazioni *oltre la fine dell'array*. In C non c'è nessun controllo sui limiti: il comportamento è indefinito.

Regole per l'inizializzazione

Quando si inizializza un array si devono rispettare le regole seguenti.

- Non si può includere un'inizializzazione nella definizione di un array a lunghezza variabile.
- Si può omettere la lunghezza dell'array nella definizione se si fornisce una lista di inizializzazione: la lunghezza è allora determinata dall'indice dell'ultimo elemento per il quale la lista contiene un inizializzatore.
- Se la definizione contiene sia una lunghezza sia una lista di inizializzazione, la lunghezza è quella specificata dall'espressione tra le parentesi quadre. Gli elementi per cui nella lista non c'è un inizializzatore vengono inizializzati a zero. Se la lista contiene più inizializzatori di quanti elementi ha l'array, quelli in eccesso vengono semplicemente ignorati.
- L'ultima virgola nella lista degli inizializzatori viene ignorata.

Di conseguenza tutte le definizioni seguenti sono equivalenti:

```
int a[4] = { 1, 2 };
int a[]  = { 1, 2, 0, 0 };
int a[]  = { 1, 2, 0, 0, };
int a[4] = { 1, 2, 0, 0, 5 };    // l'inizializzatore 5 è ignorato
```

Nell'ultima definizione l'inizializzatore 5 è ignorato; la maggior parte dei compilatori genera un warning quando c'è questo disallineamento. Se gli inizializzatori non hanno lo stesso tipo degli elementi, avviene una conversione implicita di tipo.

Inizializzare elementi specifici

I *designatori di elemento* permettono di associare gli inizializzatori a elementi specifici. Per specificare l'elemento da inizializzare si mette il suo indice tra parentesi quadre: `[espressione costante]`.

```
int a[6] = { 1, 2, [3] = 1, 2 };   // equivale a { 1, 2, 0, 1, 2, 0 }
```

Questa definizione inizializza gli elementi `a[0]` e `a[3]` con il valore 1 e gli elementi `a[1]` e `a[4]` con il valore 2. Tutti gli altri elementi ricevono il valore iniziale 0.

Il designatore deve essere un'*espressione costante intera*:

```
int three = 2;
int a[4] = { 1, [three] = 1 };
// error: expression is not an integer constant expression
```

```
const int three = 2;
int a[4] = { 1, [three] = 1 };
// ok (con alcuni compilatori): a = { 1, 0, 1, 0 }
```

15.1.6 Array come parametri di funzione

Esempio.

```c
#include <stdio.h>
void fun(int a[3]) {
    printf("%lu\n", sizeof(a));
    for (int i = 0; i < 3; ++i)
        a[i] = i;
}
int main() {
    int a[3];
    printf("%lu\n", sizeof(a));
    fun(a);
}
```

```
12
8
```

Nella `main`, `sizeof(a)` vale 12 ($3 \cdot 4$ byte). Nella funzione, invece, il parametro `int a[3]` è in realtà un puntatore: `sizeof(a)` restituisce la dimensione di `int *` (8 byte), e il compilatore avverte con `[-Wsizeof-array-argument]`: «sizeof on array function parameter will return size of 'int *' instead of 'int [3]'».

15.2 Stringhe

Una *stringa* è una sequenza continua di caratteri terminata da `'\0'`, il carattere nullo. La lunghezza di una stringa è il numero dei caratteri escluso il carattere nullo di terminazione. Le stringhe sono memorizzate in array i cui elementi hanno tipo `char`.

Si possono inizializzare array di `char` con literal stringa. Per esempio le due definizioni seguenti sono equivalenti (lunghezza della stringa 8, lunghezza dell'array 30):

```c
char str1[30] = "Let's go";
char str1[30] = { 'L', 'e', 't', '\'', 's', ' ', 'g', 'o', '\0' };
```

Un array che contiene una stringa deve essere sempre lungo almeno un elemento in più della lunghezza della stringa, per far posto al carattere nullo di terminazione.

Esempio (Array troppo corto)**.**

```c
#include <stdio.h>
int main() {
    char string[5] = "Jim Morrison";
    printf("%s\n", string);
}
```

Il compilatore avverte `initializer-string for char array is too long`; l'output è composto da caratteri casuali dopo `Jim M`, perché l'array non contiene il carattere nullo di terminazione.

Esempio (Carattere nullo nel mezzo)**.**

```c
#include <stdio.h>
int main() {
    char string[14] = {'J','i','m',' ','M','\0','o','r','r','i','s','o','n','\0'
    };
    printf("%s\n", string);
}
```

```
Jim M
```

La stampa si ferma al primo `'\0'`.

Se si definisce un array di caratteri senza lunghezza esplicita e lo si inizializza con un literal stringa, l'array creato è lungo un elemento *in più* della lunghezza della stringa:

```c
char str2[] = " to London!";    // lunghezza della stringa: 11
                                // lunghezza dell'array: 12
```

15.3 Array multidimensionali

Un array multidimensionale in C è semplicemente un array i cui elementi sono a loro volta array. Gli elementi di un array a n dimensioni sono array a $(n-1)$ dimensioni. Per esempio, ogni elemento di un array a due dimensioni è un array a una dimensione.

La dichiarazione di un array multidimensionale ha una coppia di parentesi quadre per ogni dimensione:

```c
char screen[10][40][80];    // un array a tre dimensioni
```

Ha 10 elementi, da `screen[0]` a `screen[9]`; ognuno è un array a due dimensioni, formato a sua volta da 40 array a una dimensione di 80 caratteri. In totale $10 \cdot 40 \cdot 80 = 32\,000$ elementi. Lo standard raccomanda alle implementazioni di accettare almeno 256 dimensioni (ISO 14882, B.2), ma possono supportarne meno o più.

15.3.1 Matrici

Gli array bidimensionali sono detti anche *matrici*. Sono molto usati e meritano uno sguardo più attento. La matrice `mat` della definizione seguente ha tre righe e cinque colonne (conviene ragionare in termini di righe e colonne):

```c
float mat[3][5];
```

`mat[0]`, `mat[1]` e `mat[2]` sono le righe della matrice; ognuna è un array di cinque elementi `float`. In totale 15 elementi `float`.

```
for (int row = 0; row < 3; ++row)
    for (int col = 0; col < 5; ++col)
        mat[row][col] = row + (float) col / 10;
```

15.3.2 Inizializzazione

Si possono inizializzare array multidimensionali con una lista di inizializzazione secondo le regole già viste:

```
int a3d[2][2][3] = { { { 1, 0, 0 }, { 4, 0, 0 } }, { { 7, 8, 0 }, { 0, 0, 0 } }
    };
int a3d[][2][3]  = { { { 1 }, { 4 } }, { { 7, 8 } } };
```

Nella seconda non occorre specificare che la prima dimensione ha ampiezza 2, perché la lista più esterna contiene due inizializzatori.

Si possono anche omettere alcune parentesi graffe. Se una coppia di graffe contiene più inizializzatori del numero di elementi della dimensione corrispondente, quelli in eccesso sono associati all'elemento successivo nella sequenza di memorizzazione:

```
int a3d[2][2][3] = { { 1, 0, 0, 4 }, { 7, 8 } };
int a3d[2][2][3] = { 1, 0, 0, 4, 0, 0, 7, 8 };
```

Si possono usare anche i designatori di elemento:

```
int a3d[2][2][3] = { 1, [0][1][0] = 4, [1][0][0] = 7, 8 };
```

15.4 Riferimenti

Sul libro: sezioni 6.1–6.5, 6.11; stringhe: capitoli 8 e 9.

16 Funzioni e funzione main

Lezione 16 – 3 novembre 2026

16.1 Funzioni

16.1.1 Introduzione e main

Tutte le istruzioni di un programma C sono contenute in funzioni. Il C è un linguaggio procedurale: ogni funzione svolge un certo compito. Un nome speciale di funzione è `main()`: la funzione con questo nome è la prima a essere eseguita quando il programma parte. Il primo comando eseguito nel programma è il primo comando della `main` (dall'alto verso il basso). Tutte le altre funzioni sono sottoprogrammi della `main()` e possono avere i nomi che si vuole.

Ogni funzione è definita esattamente una volta. Un programma può dichiarare e chiamare una funzione tante volte quante ne servono.

16.1.2 Definizione di funzione

La definizione di una funzione consiste in un'*intestazione* (o dichiaratore) e in un *blocco* della funzione. L'intestazione specifica il nome della funzione, il tipo del valore restituito e i tipi e i nomi dei parametri, se ce ne sono. Le istruzioni nel blocco specificano che cosa fa la funzione.

```
tipo nome_funzione (tipo nome_parametro1, tipo nome_parametro2, ...) {
    // istruzioni;
}
```

Osservazione. Scrivere sempre il tipo del valore restituito: se non è specificato è `int`.

16.1.3 Dichiarazione di funzione

Dichiarando una funzione prima di usarla si informa il compilatore del suo tipo: la dichiarazione descrive cioè l'*interfaccia* della funzione. Una dichiarazione definisce

- il tipo della funzione;
- il numero e i tipi dei suoi parametri;
- il tipo del valore restituito.

Gli identificatori dei parametri in una dichiarazione di funzione sono facoltativi. Se si scrivono i nomi, il loro scope termina con il prototipo stesso. Le due forme sono entrambe valide:

```
tipo nome_funzione (tipo nome_parametro1, tipo nome_parametro2, ...);
tipo nome_funzione (tipo, tipo, ...);
```

16.1.4 Scope degli identificatori

- *Block scope*: gli identificatori dichiarati in un blocco hanno block scope. Si può usare un tale identificatore solo dalla sua dichiarazione fino alla fine del blocco più piccolo che contiene la dichiarazione. Il blocco più piccolo che la contiene è spesso, ma non necessariamente, il corpo di una definizione di funzione. In C11 le dichiarazioni non devono essere necessariamente poste prima di tutte le istruzioni nel blocco di una funzione. Anche i nomi dei parametri nell'intestazione di una definizione di funzione hanno block scope, e sono validi nel blocco corrispondente della funzione.
- *Function prototype scope*: i nomi dei parametri in un prototipo di funzione hanno function prototype scope. Poiché questi nomi non hanno significato al di fuori del prototipo stesso, hanno valore solo di commento e si possono anche omettere.

Esempio.

```
int p;
int fun1(int q);
int fun2(int r)
{
    int s = r + 3;
    while (r != s)
        r--;
```

```
}
```

`p`, `fun1` e `fun2` sono visibili in tutto il file (da dove sono dichiarati); `q` ha scope il solo prototipo; `r` e `s` hanno scope il blocco di `fun2`.

16.1.5 Parametri di una funzione

I parametri di una funzione sono normali variabili locali. Il programma li crea e li inizializza con i valori dei corrispondenti argomenti quando avviene una chiamata di funzione. Il loro scope è il blocco della funzione (block scope).

```
long double factorial(unsigned int n)
{
    long double f = 1;
    while (n > 1)
        f *= n--;
    return f;
}
```

16.1.6 Dichiarazione e definizione

Una definizione di funzione è anche una dichiarazione di funzione. Le due versioni seguenti sono equivalenti: nella prima `fun2` è dichiarata prima della `main` e definita dopo; nella seconda è definita prima della `main`, e la definizione fa anche da dichiarazione.

```
int fun2(int);
int main() {
    int a = 5;
    a = fun2(a);
}
int fun2(int r) {
    int s = r + 5;
    while (r != s)
        r--;
    return r;
}
```

```
int fun2(int r) {
    int s = r + 5;
    while (r != s)
        r--;
    return r;
}
int main() {
    int a = 5;
    a = fun2(a);
}
```

In sintesi:

- prima di chiamare (usare) una funzione bisogna dichiararla;
- si possono dichiarare tutte le funzioni all'inizio del file e definirle in un ordine qualunque più sotto nello stesso file; oppure si possono evitare le dichiarazioni in testa, ma allora vanno definite *prima* di essere chiamate;
- una funzione deve anche essere definita, altrimenti non si sa che cosa fare quando viene chiamata;
- nello stesso file non si possono avere due definizioni di funzioni con la stessa intestazione e lo stesso identificatore (nome).

Esempio (Dichiarazioni e ordine delle definizioni)**.** Questa versione è corretta: `fun1` e `fun2` sono dichiarate in testa.

```
1  int fun1(int);
2  int fun2(int);
3  int main() {
4      int a = 5;
5      a = fun1(a);
6  }
7  int fun1(int q) {
8      return (fun2(q) + 1);
9  }
10 int fun2(int r) {
11     int s = r + 5;
12     while (r != s)
13         r--;
14     return r;
15 }
```

Questa invece è *sbagliata*: `fun1` chiama `fun2`, che non è stata dichiarata prima di quel punto (e non è definita prima).

```
1  int fun1(int);
2  int main() {
3      int a = 5;
4      a = fun1(a);
5  }
6  int fun1(int q) {
7      return (fun2(q) + 1);   // errore: fun2 non dichiarata
8  }
9  int fun2(int r) {
10     int s = r + 5;
11     while (r != s)
12         r--;
13     return r;
14 }
```

Si può correggere dichiarando `fun2` in testa, oppure definendola prima di `fun1`.

16.1.7 return

L'istruzione `return` termina l'esecuzione della funzione corrente e salta indietro al punto in cui la funzione era stata chiamata:

```
return [espressione];
```

L'espressione viene valutata e il risultato viene restituito al chiamante come valore della chiamata di funzione. Questo valore restituito viene convertito al tipo restituito dalla funzione, se necessario. Una funzione può contenere un numero qualunque di istruzioni `return`; il primo `return` eseguito fa uscire dalla funzione. `return` è un'istruzione di salto incondizionato (come `goto`).

Esempio.

```c
// Restituisce il minore di due argomenti interi.
int min(int a, int b)
{
    if (a < b)
        return a;
    else
        return b;
}
int main()
{
    int a = 5;
    int b = 6;
    int c = min(a, b);
    printf("%d", c);
}
```

```
5
```

Un `return` senza espressione può essere usato solo in una funzione di tipo `void`; anzi, queste funzioni non hanno nemmeno bisogno di un'istruzione `return`. Se in una funzione non si incontra nessun `return`, il flusso del programma torna al chiamante quando si raggiunge la fine del blocco della funzione.

16.2 La funzione main

Si può definire la funzione `main()` in una delle due forme seguenti:

```c
int main(void) { /* ... */ }                    // nessun parametro, restituisce
    int
int main(int argc, char *argv[]) { /* ... */ } // due parametri di tipo int e
    char **, restituisce int
```

16.2.1 Parametri della main

I parametri `argc` e `argv` (ai quali si possono dare altri nomi) rappresentano gli *argomenti sulla riga di comando* del programma. Funzionano così:

- `argc` (da *argument count*) è 0 oppure il numero dei token stringa nella riga di comando che ha avviato il programma. Il nome del programma stesso è incluso in questo conteggio;
- `argv` (da *argument vector*) è un array di puntatori a `char` che puntano ai singoli token stringa ricevuti sulla riga di comando. Il numero di elementi di questo array è uno in più del valore di `argc`: l'ultimo elemento, `argv[argc]`, è sempre un puntatore nullo;
- la prima stringa, `argv[0]`, contiene il nome con cui è stato invocato il programma. Se l'ambiente di esecuzione non fornisce il nome del programma, la stringa è vuota;
- se `argc` è maggiore di 1, le stringhe da `argv[1]` a `argv[argc - 1]` contengono gli argomenti sulla riga di comando del programma.

Esempio (Un esempio classico)**.**

```
#include <stdio.h>
int main(int argc, char *argv[])
{
    if (argc == 0)
        puts("No command line available.");
    else {
        printf("The program now running: %s\n", argv[0]);  // nome del programma
        if (argc == 1)
            puts("No arguments received on the command line.");
        else {
            puts("The command line arguments:");
            for (int i = 1; i < argc; ++i)    // un argomento per riga
                puts(argv[i]);
        }
    }
}
```

Se su un sistema Unix si lancia il programma con la riga di comando

```
$ ./args one two "and three"
```

l'output è

```
The program now running: ./args
The command line arguments:
one
two
and three
```

16.2.2 Valore restituito dalla main

Il valore restituito dalla `main` viene passato al processo padre che l'ha eseguita (per esempio la shell). Il valore restituito dalla `main` dovrebbe indicare come è terminato il programma: l'uscita normale è generalmente rappresentata dal valore 0; una terminazione anomala è di solito segnalata da un valore diverso da zero.

16.3 Come vengono eseguite le funzioni

16.3.1 Uscire da una funzione

Se il programma raggiunge un'istruzione `return` o la parentesi graffa di chiusura `}` del blocco della funzione, l'esecuzione della funzione termina e il programma torna alla funzione chiamante. Se il programma «esce dal fondo» della funzione raggiungendo la graffa di chiusura, il valore restituito al chiamante è *indefinito*. Per questo motivo bisogna usare un `return` per terminare qualunque funzione che non abbia tipo `void`. Il valore dell'espressione di `return` viene restituito alla funzione chiamante.

16.3.2 Chiamare una funzione

L'istruzione che esegue una funzione, la *chiamata di funzione*, consiste nel nome della funzione e nell'operatore `()`. Per esempio, l'istruzione seguente chiama la funzione `maximum()` per calcolare il massimo di due numeri:

```
1 maximum(a, b);
```

Il programma prima alloca lo spazio di memoria per i parametri, poi copia i valori degli argomenti nelle locazioni corrispondenti. Quindi salta all'inizio della funzione, e l'esecuzione della funzione comincia con la prima definizione di variabile o istruzione del blocco.

Esempio.

```
1  #include <stdio.h>
2  int sum(int p, int q) {
3      int r = 0;
4      r = p + q;
5      return r;
6  }
7  int main(void) {
8      int i = 3, j = 5, s = 0;
9      s = sum(i, j);
10     printf("%d\n", s);
11     return 0;
12 }
```

```
8
```

Al momento della chiamata, `p` e `q` ricevono copie dei valori di `i` (3) e `j` (5): `i` e `j` sono passati *per valore*. `r` vale prima 0 e poi 8, che viene restituito e assegnato a `s`.

16.3.3 Parametri formali e attuali

Quando una funzione viene chiamata, i valori (le espressioni) passati nella chiamata si chiamano *argomenti* o *parametri attuali* (i due termini hanno lo stesso significato). Al momento della chiamata, ogni parametro attuale viene assegnato al corrispondente *parametro formale* nella definizione della funzione. Nell'esempio precedente `p` e `q` sono i parametri formali; i valori di `i` e `j` sono i parametri attuali.

16.3.4 Allocazione delle variabili

L'*allocazione di memoria* significa che una parte della memoria viene riservata, per esempio per una variabile. Una *variabile automatica* è una variabile locale che viene allocata e deallocata automaticamente quando il flusso del programma entra nel suo scope e ne esce. Per *deallocazione* si intende che quella parte di memoria non è più riservata per quella variabile. Le variabili `p` e `q` vengono allocate quando la funzione è chiamata e deallocate quando se ne esce.

16.4 Overloading di funzioni

L'*overloading* è una caratteristica di un linguaggio di programmazione che permette di avere più funzioni con lo stesso nome ma con firme diverse (tipo dei parametri e tipo restituito). È presente nella maggior parte dei linguaggi orientati agli oggetti, come C++ e Java. **Il C non supporta questa caratteristica.** Per fare l'overloading di una funzione in C++ la si può definire più volte, ciascuna identificata in modo univoco dal tipo degli argomenti che accetta; il tipo restituito non viene considerato.

Esempio.

```
1  #include <stdio.h>
2  int min(int x, int y) {
3      return (x < y ? x : y);
4  }
5  float min(float x, float y) {
6      return (x < y ? x : y);
7  }
8  int main() {
9      int a = min(4, 1);
10     float b = min(4.56F, 1.23F);
11 }
```

Compilando con `gcc` si ottiene l'errore `conflicting types for 'min'`; compilando lo stesso file come C++ (`g++`) funziona.

16.5 Riferimenti

Sul libro: sezioni 5.1–5.6, 15.3, 7.4.